

Controller Area Network (DCAN) Module

This chapter describes the controller area network (DCAN) module.

NOTE: This chapter describes a superset implementation of the DCAN module. Consult your device-specific datasheet to identify the applicability of the DMA-related features, the number of instantiations of the DCAN IP, and the number of mailboxes supported on your specific device being used.

Topic	Page
26.1 Overview.....	1329
26.2 CAN Blocks.....	1330
26.3 CAN Bit Timing.....	1332
26.4 CAN Module Configuration	1336
26.5 Message RAM.....	1338
26.6 Message Interface Register Sets.....	1342
26.7 Message Object Configurations.....	1345
26.8 Message Handling.....	1347
26.9 CAN Message Transfer	1352
26.10 Interrupt Functionality.....	1353
26.11 Global Power-Down Mode.....	1355
26.12 Local Power-Down Mode	1356
26.13 GPIO Support.....	1357
26.14 Test Modes	1358
26.15 Parity Check Mechanism	1362
26.16 Debug/Suspend Mode	1363
26.17 DCAN Control Registers.....	1363

26.1 Overview

The Controller Area Network is a high-integrity, serial, multi-master communication protocol for distributed real-time applications. This CAN module is implemented according to ISO 11898-1 and is suitable for industrial, automotive and general embedded communications.

26.1.1 Features

The DCAN module provides the following features:

Protocol

- Supports CAN protocol version 2.0 part A, B

Speed

- Bit rates up to 1 MBit/s

MailBox

- Configurable Message objects
- Individual identifier masks for each message object
- Programmable FIFO mode for message objects

High Speed MailBox Access

- DMA access to Message RAM.

Power

- Global power down and wakeup support
- Local power down and wakeup support

Debug

- Suspend mode for debug support
- Programmable loop-back modes for self-test operation
- Direct access to Message RAM in test mode
- Supports Two interrupt lines - Level 0 and Level 1

Others

- Automatic Message RAM initialization
- Automatic bus on after Bus-Off state by a programmable 32-bit timer
- CAN Rx / Tx pins configurable as general purpose IO pins
- Software module reset
- Message RAM parity check mechanism
- Dual clock source to reduce jitter

26.1.2 Functional Description

The CAN protocol is an ISO standard (ISO 11898) for serial data communication. This protocol uses Non-Return To Zero (NRZ) with bit-stuffing. And the communication is carried over a two-wire balanced signaling scheme.

The DCAN data communication happens through the CAN_TX and CAN_RX pins. An additional transceiver hardware is required for the connection to the physical layer (CAN bus) CAN_High and CAN_Low.

The DCAN register set can be accessed directly by the CPU. These registers are used to control and configure the CAN module and the Message RAM.

Individual CAN message objects should be configured for communication over a CAN network. The message objects and identifier masks are stored in the Message RAM.

The CAN module internally handles functions such acceptance filtering, transfer of messages from and to the Message RAM, handling of transmission requests as well as the generation of interrupts or DMA requests.

26.2 CAN Blocks

The DCAN Module, shown in [Figure 26-1](#), comprises of the following basic blocks.

26.2.1 CAN Core

The CAN Core consists of the CAN Protocol Controller and the Rx/Tx Shift Register. It handles all ISO 11898-1 protocol functions.

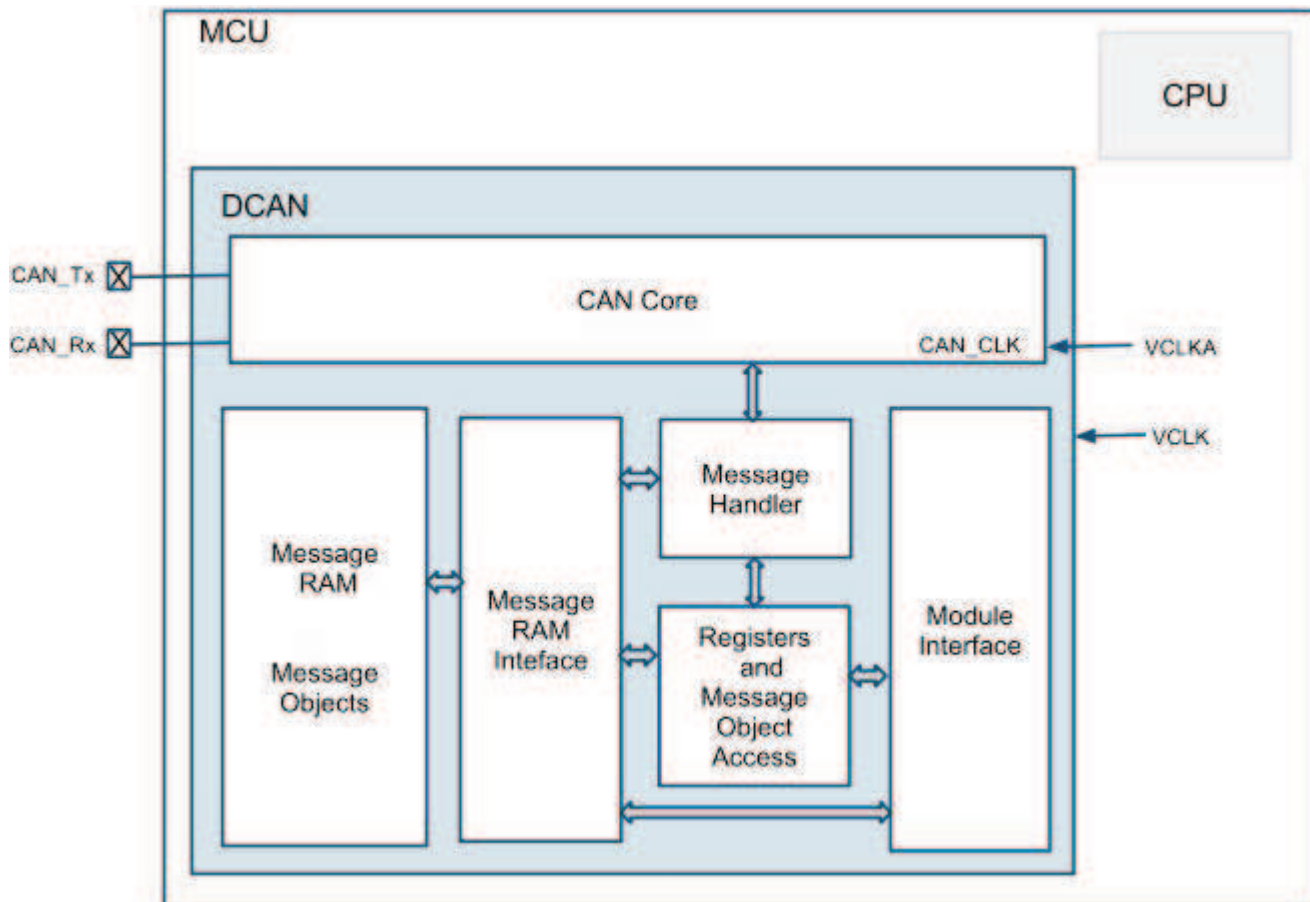
26.2.2 Message RAM

The DCAN Message RAM enables storage of CAN messages.

26.2.3 Message Handler

The Message Handler is a state machine that controls the data transfer between the single ported Message RAM and the CAN Core's Rx/Tx Shift Register. It also handles acceptance filtering and the interrupt/DMA request generation as programmed in the control registers.

Figure 26-1. Block Diagram



26.2.4 Message RAM Interface

The Interface Register sets control the CPU read and write accesses to the Message RAM.

There are three interface registers IF1, IF2, and IF3.

IF1 and IF2 Interface Registers sets for read and write access.

IF3 Interface Register set for read access only.

The Interface Registers have the same word-length as the Message RAM.

Additional information can be found in [Section 26.6](#).

26.2.5 Register and Message Object Access

During normal operation, data consistency of the message objects is guaranteed by indirectly accessing the message objects through the interface registers IF1 and IF2.

In order to be able to perform tests on the message object memory, a dedicated test mode has been implemented, that allows direct access by either the CPU or DMA. During normal operation direct access has to be avoided.

26.2.6 Dual Clock Source

Two clock domains are provided to the DCAN module:

1. VCLK - The peripheral synchronous clock domain as the general module clock source.
2. VCLKA - The peripheral asynchronous clock source domain provided to the CAN core as clock source (CAN_CLK) for generating the CAN Bit Timing.

If a frequency modulated clock output from FMPLL is used as the VCLK source, then VCLKA should be derived from an unmodulated clock source (for example, OSCIN source).

The clock source for VCLKA is selected by the Peripheral Asynchronous Clock Source Register in the system module.

Both clock domains can be derived from the same clock source (so that VCLK = VCLKA). However, if frequency modulation in the FMPLL is enabled (spread spectrum clock), then due to the high precision clocking requirements of the CAN Core, the FMPLL clock source should not be used for VCLKA. Alternatively, a separate clock without any modulation (for example, derived directly from the OSCIN clock) should be used for VCLKA.

Please refer to the system module reference guide and the device datasheet for more information how to configure the relevant clock source registers in the system module.

Between the two clock domains, a synchronization mechanism is implemented in the DCAN module in order to ensure correct data transfer.

NOTE: If the dual clock functionality is used, then VCLK must always be higher or equal to CAN_CLK (derived from the asynchronous clock source), in order to achieve a stable functionality of the DCAN. Here also the frequency shift of the modulated VCLK has to be considered:

$$f_{0, VCLK} \pm \Delta f_{FM, VCLK} \geq f_{CANCLK}$$

The CAN Core has to be programmed to at least 8 clock cycles per bit time. To achieve a transfer rate of 1 MBaud when using the asynchronous clock domain as the clock source for CAN_CLK, an oscillator frequency of 8MHz or higher has to be used.

26.3 CAN Bit Timing

The DCAN supports bit rates between less than 1 kBit/s and 1000 kBit/s.

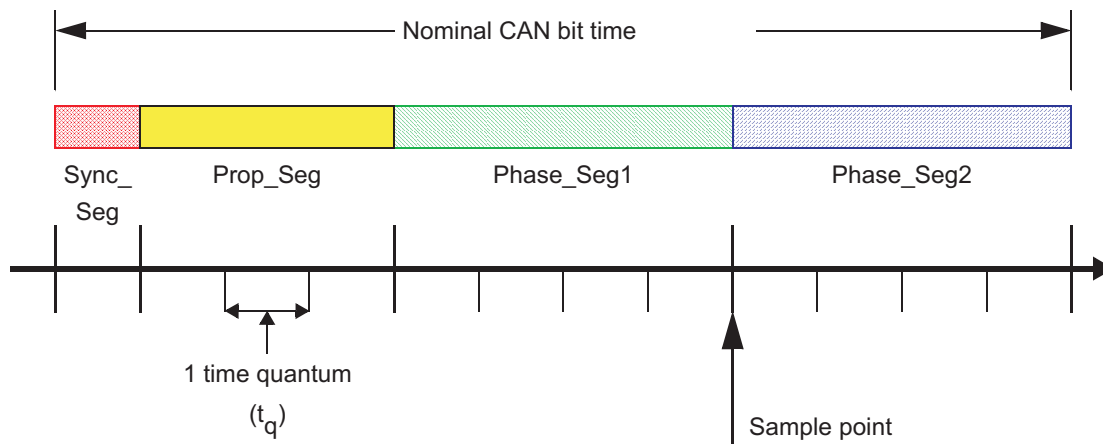
Each member of the CAN network has its own clock generator, typically derived from a crystal oscillator. The Bit timing parameters can be configured individually for each CAN node, creating a common Bit rate even though the CAN nodes' oscillator periods (f_{osc}) may be different.

26.3.1 Bit Time and Bit Rate

According to the CAN specification, the Bit time is divided into four segments (see Figure 26-2):

- Synchronization Segment (Sync_Seg)
- Propagation Time Segment (Prop_Seg)
- Phase Buffer Segment 1 (Phase_Seg1)
- Phase Buffer Segment 2 (Phase_Seg2)

Figure 26-2. Bit Timing



Each segment consists of a specific number of time quanta. The length of one time quantum, (t_q), which is the basic time unit of the bit time, is given by the CAN_CLK and the Baud Rate Prescalers (BRPE and BRP). With these two Baud Rate Prescalers combined, divider values from 1 to 1024 can be programmed:

$$t_q = \text{Baud Rate Prescaler} / \text{CAN_CLK}$$

Apart from the fixed length of the synchronization segment, these numbers are programmable.

Table 26-1 describes the minimum programmable ranges required by the CAN protocol. A given bit rate may be met by different Bit time configurations.

NOTE: For proper functionality of the CAN network, the physical delay times and the oscillator's tolerance range have to be considered.

Table 26-1. Parameters of the CAN Bit Time

Parameter	Range	Remark
Sync_Seg	1 t_q (fixed)	Synchronization of bus input to CAN_CLK
Prop_Seg	[1 ... 8] t_q	Compensates for the physical delay times
Phase_Seg1	[1 ... 8] t_q	May be lengthened temporarily by synchronization
Phase_Seg2	[1 ... 8] t_q	May be shortened temporarily by synchronization
Synchronization Jump Width (SJW)	[1 ... 4] t_q	May not be longer than either Phase Buffer Segment

26.3.1.1 Synchronization Segment

The Synchronization Segment (Sync_Seg) is the part of the bit time where edges of the CAN bus level are expected to occur. If an edge occurs outside of Sync_Seg, its distance to the Sync_Seg is called the phase error of this edge.

26.3.1.2 Propagation Time Segment

This part of the bit time is used to compensate physical delay times within the CAN network. These delay times consist of the signal propagation time on the bus and the internal delay time of the CAN nodes.

26.3.1.3 Phase Buffer Segments and Synchronization

The Phase Buffer Segments (Phase_Seg1 and Phase_Seg2) and the Synchronization Jump Width (SJW) are used to compensate for the oscillator tolerance.

The Phase Buffer Segments surround the sample point. The Phase Buffer Segments may be lengthened or shortened by synchronization.

The Synchronization Jump Width (SJW) defines how far the resynchronizing mechanism may move the sample point inside the limits defined by the Phase Buffer Segments to compensate for edge phase errors.

Synchronizations occur on edges from recessive to dominant. Their purpose is to control the distance between edges and sample points.

Edges are detected by sampling the actual bus level in each time quantum and comparing it with the bus level at the previous sample point. A synchronization may be done only if a recessive bit was sampled at the previous sample point and if the actual time quantum's bus level is dominant.

An edge is synchronous if it occurs inside of Sync_Seg, otherwise its distance to the Sync_Seg is the edge phase error, measured in time quanta. If the edge occurs before Sync_Seg, the phase error is negative, else it is positive.

26.3.1.4 Oscillator Tolerance Range

With the introduction of CAN protocol version 1.2, the option to synchronize on edges from dominant to recessive became obsolete. Only edges from recessive to dominant are considered for synchronization. The protocol update to version 2.0 (A and B) had no influence on the oscillator tolerance.

The tolerance range df for an oscillator's frequency f_{osc} around the nominal frequency f_{nom} with:

$$(1-df) \times f_{nom} \leq f_{osc} \leq (1+df) \times f_{nom} \quad (36)$$

depends on the proportions of Phase_Seg1, Phase_Seg2, SJW, and the bit time. The maximum tolerance df is defined by two conditions (both shall be met):

$$I: df \leq \frac{\min(\text{Phase_Seg1}, \text{Phase_Seg2})}{[2 \times (13 \times \text{bit_time} - \text{Phase_Seg2})]}$$

$$II: df \leq \frac{\text{SJW}}{20 \times \text{bit_time}} \quad (37)$$

It has to be considered that SJW may not be larger than the smaller of the Phase Buffer Segments and that the Propagation Time Segment limits that part of the bit time that may be used for the Phase Buffer Segments.

The combination Prop_Seg = 1 and Phase_Seg1 = Phase_Seg2 = SJW = 4 allows the largest possible oscillator tolerance of 1.58%. This combination with a Propagation Time Segment of only 10% of the bit time is not suitable for short bit times; it can be used for bit rates of up to 125 kBit/s (bit time = 8 μ s) with a bus length of 40 m.

26.3.2 DCAN Bit Timing Registers

In the DCAN, the bit timing configuration is programmed in two register bytes, additionally a third byte for a baud rate prescaler extension of 4 bits (BREP) is provided. The sum of Prop_Seg and Phase_Seg1 (as TSEG1) is combined with Phase_Seg2 (as TSEG2) in one byte, SJW and BRP (plus BRPE in third byte) are combined in the other byte

In this bit timing register, the components TSEG1, TSEG2, SJW and BRP have to be programmed to a numerical value that is one less than its functional value; so instead of values in the range of [1... n], values in the range of [0 ... n-1] are programmed. That way, SJW (functional range of [1 ... 4]) is represented by only two bits.

Therefore the length of the Bit time is (programmed values) $[TSEG1 + TSEG2 + 3] t_q$ or (functional values) $[Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2] t_q$.

The data in the Bit Timing Register (BTR) is the configuration input of the CAN protocol controller. The Baud Rate Prescaler (configured by BRPE/BRP) defines the length of the time quantum (the basic time unit of the bit time); the Bit Timing Logic (configured by TSEG1, TSEG2, and SJW) defines the number of time quanta in the bit time.

26.3.2.1 Calculation of the Bit Timing Parameters

Usually, the calculation of the bit timing configuration starts with a desired bit rate or bit time. The resulting Bit time ($1 / \text{Bit rate}$) must be an integer multiple of the CAN clock period.

NOTE: 8 MHz is the minimum CAN clock frequency required to operate the DCAN at a bit rate of 1 MBit/s.

The bit time may consist of 8 to 25 time quanta. The length of the time quantum t_q is defined by the Baud Rate Prescaler with $t_q = (\text{Baud Rate Prescaler}) / \text{CAN_CLK}$. Several combinations may lead to the desired bit time, allowing iterations of the following steps.

First part of the bit time to be defined is the Prop_Seg. Its length depends on the delay times measured in the system. A maximum bus length as well as a maximum node delay has to be defined for expandible CAN bus systems. The resulting time for Prop_Seg is converted into time quanta (rounded up to the nearest integer multiple of t_q).

The Sync_Seg is 1 t_q long (fixed), leaving $(\text{bit time} - \text{Prop_Seg} - 1) t_q$ for the two Phase Buffer Segments. If the number of remaining t_q is even, the Phase Buffer Segments have the same length, $\text{Phase_Seg2} = \text{Phase_Seg1}$, else $\text{Phase_Seg2} = \text{Phase_Seg1} + 1$.

The minimum nominal length of Phase_Seg2 has to be regarded as well. Phase_Seg2 may not be shorter than any CAN controller's Information Processing Time in the network, which is device dependent and can be in the range of [0 ... 2] t_q .

The length of the Synchronization Jump Width is set to its maximum value, which is the minimum of 4 and Phase_Seg1.

If more than one configurations are possible to reach a certain Bit rate, it is recommended to choose the configuration that allows the highest oscillator tolerance range.

CAN nodes with different clocks require different configurations to come to the same bit rate. The calculation of the propagation time in the CAN network, based on the nodes with the longest delay times, is done once for the whole network.

The CAN system's oscillator tolerance range is limited by the node with the lowest tolerance range.

The calculation may show that bus length or bit rate have to be decreased or that the oscillator frequencies' stability has to be increased in order to find a protocol compliant configuration of the CAN bit timing.

The resulting configuration is written into the Bit Timing Register:

Tseg2 = Phase_Seg2-1
Tseg1 = Phase_Seg1+ Prop_Seg-1
SJW = SynchronizationJumpWidth-1
BRP = Prescaler-1

26.3.2.2 Example for Bit Timing at High Baudrate

In this example, the frequency of CAN_CLK is 10 MHz, BRP is 0, the bit rate is 1 MBit/s.

t_q	100	ns	=	t_{CAN_CLK}	
delay of bus driver	60	ns			
delay of receiver circuit	40	ns			
delay of bus line (40m)	220	ns			
t_{Prop}	700	ns	=	$INT(2 \times \text{delays} + 1) = 7 \times t_q$	
t_{SJW}	100	ns	=	$1 \times t_q$	
t_{TSeg1}	800	ns	=	$t_{Prop} + t_{SJW}$	
t_{TSeg2}	100	ns	=	Information Processing Time + $1 \times t_q$	
$t_{Sync-Seg}$	100	ns	=	$1 \times t_q$	
bit time	1000	ns	=	$t_{Sync-Seg} + t_{TSeg1} + t_{TSeg2}$	
tolerance for CAN_CLK	1.58	%	=	$\frac{\min(TSeg1, TSeg2)}{[2 \times (13 \times \text{bit_time} - TSeg2)]}$	(38)

$$= \frac{0.1 \mu s}{[2 \times (13 \times 1 \mu s - 0.1 \mu s)]} \quad (39)$$

$$= 0.38\%$$

In this example, the concatenated bit time parameters are $(1-1)_3$ & $(8-1)_4$ & $(1-1)_2$ & $(1-1)_6$, so the Bit Timing Register is programmed to 0000 0700h.

26.3.2.3 Example for Bit Timing at Low Baudrate

In this example, the frequency of CAN_CLK is 2 MHz, BRP is 1, the bit rate is 100 KBit/s.

t_q	1	μs	=	$2 \times t_{CAN_CLK}$	
delay of bus driver	200	ns			
delay of receiver circuit	80	ns			
delay of bus line (40m)	220	ns			
t_{Prop}	1	μs	=	$1 \times t_q$	
t_{SJW}	4	μs	=	$4 \times t_q$	
t_{TSeg1}	5	μs	=	$t_{Prop} + t_{SJW}$	
t_{TSeg2}	3	μs	=	Information Processing Time + $3 \times t_q$	
$t_{Sync-Seg}$	1	μs	=	$1 \times t_q$	
bit time	9	μs	=	$t_{Sync-Seg} + t_{TSeg1} + t_{TSeg2}$	
tolerance for CAN_CLK	0.43	%	=	$\frac{\min(TSeg1, TSeg2)}{[2 \times (13 \times \text{bit_time} - TSeg2)]}$	(40)

$$= \frac{3 \mu s}{[2 \times (13 \times 9 \mu s - 3 \mu s)]} \quad (41)$$

$$= 1.32\%$$

In this example, the concatenated bit time parameters are $(3-1)_3$ & $(5-1)_4$ & $(4-1)_2$ & $(2-1)_6$, so the Bit Timing Register is programmed to 0000 24C1h.

26.4 CAN Module Configuration

After a hardware reset all CAN protocol functions are disabled. The CAN module must be initialized and configured before it can participate on the CAN bus.

26.4.1 DCAN RAM Initialization through Hardware

To start with a clean DCAN RAM ,the complete DCAN RAM can be initialized with zeros and the ecc/parity bits set accordingly by configuring the following registers in the system module:

1. Memory Hardware Initialization Global Control Register (MINITGCR)
2. Memory Initialization Enable Register (MSINENA)

For more details on RAM hardware initialization support, refer to the system module reference guide.

26.4.2 CAN Module Initialization

To initialize the CAN Controller, you have to set up the CAN Bit timing and those message objects that have to be used for CAN communication. Message objects that are not needed, can be deactivated.

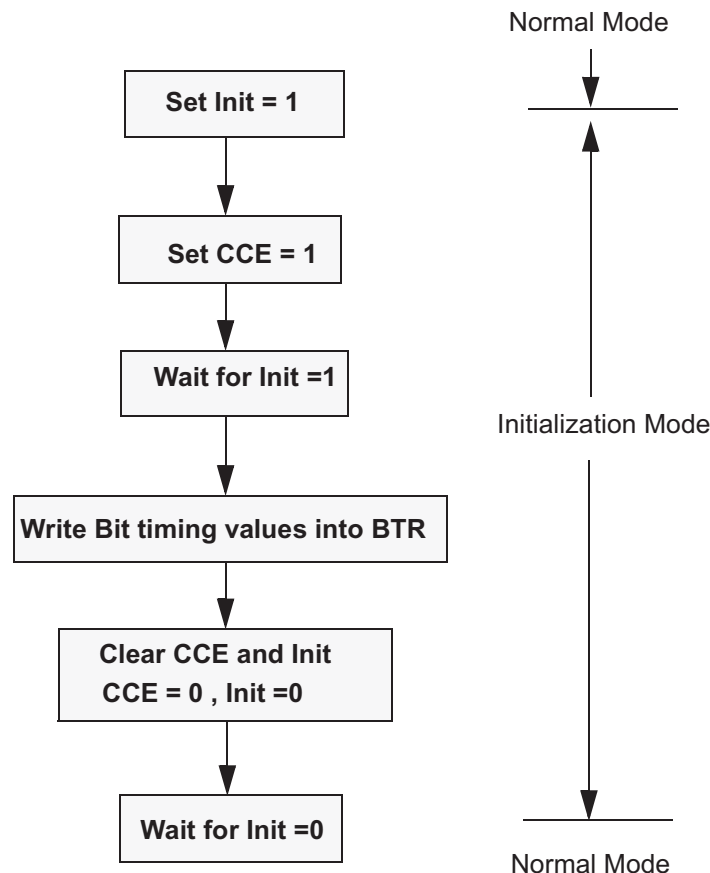
So the two critical steps are:

1. Configuration of CAN Bit Timings
2. Configuration of Message Objects

26.4.2.1 Software Configuration of CAN Bit Timings

This step involves configuring the CAN baud rate register with the calculated CAN bit timing value. The calculation procedure of CAN bit timing values for BTR register are mentioned in [Section 26.3](#). Refer to [Figure 26-3](#) for CAN bit timing software configuration flow.

Figure 26-3. CAN Bit-timing Configuration



Step 1: Enter “initialization mode” by setting the Init (Initialization) bit in the CAN Control Register.

While the Init bit is set, the message transfer from and to the CAN bus is stopped, and the status of the CAN_TX output is recessive (high).

The CAN error counters are not updated. Setting the Init bit does not change any other configuration register.

Also note that the CAN module is also in initialization mode on hardware reset and during Bus-Off.

Step 2: Set the CCE (Configure Change Enable) bit in the CAN Control Register.

The access to the Bit Timing Register for the configuration of the Bit timing is enabled when both Init and CCE bits in the CAN Control Register are set.

Step 3: Wait for the Init bit to get set. This would make sure that the module has entered “Initialization mode”.

Step 4: Write the Bit-Timing values into the Bit-Timing Register (BTR).

Refer to [Section 26.3.2.1](#) for BTR value calculation for a given bit-timing.

Step 5: Clear the CCE bit followed by Init bit.

Step 6: Wait for the Init bit to clear. This would make sure that the module has come out of “initialization mode”.

After step 6 (Init bit cleared), the module will attempt a synchronization on the CAN bus, provided that the BTR settings are meeting the CAN bus parameters.

NOTE: The module would not come out of the “initialization mode” if any incorrect BTR values are written in step 4.

26.4.2.2 Configuration of Message Objects

The whole Message RAM should be configured before putting the CAN into operation. All the message objects are deactivated by default. The user should configure the message object that are to be used to a particular identifier. The user can change the configuration of any message object or deactivate it when required.

The message objects can be configured only through the Interface registers (IFx) and the CPU does not have direct access to the message object (Message RAM) when DCAN is in operation.

To configure the message objects, the user must know about:

1. The message object structure ([Section 26.5](#))
2. The interface register set (IFx) ([Section 26.6](#))

NOTE: The message objects initialization is independent of the bit-timing configuration procedure.

26.5 Message RAM

The DCAN Message RAM contains message objects and parity bits for the message objects.

26.5.1 Structure of Message Objects

Figure 26-4 shows the structure of a message object.

The grayed fields are those parts of the message object that are represented in dedicated registers. For example, the transmit request flags of all message objects are represented in centralized transmit request registers.

Figure 26-4. Structure of a Message Object

Message Object												
UMask	Msk[28:0]	MXtd	MDir	EoB	unused	NewDat	MsgLst	RxIE	TxIE	IntPnd	RmtEn	TxRqst
MsgVal	ID[28:0]	Xtd	Dir	DLC[3:0]	Data 0	Data 1	Data 2	Data 3	Data 4	Data 5	Data 6	Data 7

Table 26-2. Message Object Field Descriptions

Name	Value	Description
MsgVal		Message valid
	0	The message object is ignored by the Message Handler.
	1	The message object is to be used by the Message Handler. Note: The CPU should reset the MsgVal bit of all unused Messages Objects during the initialization before it resets bit Init in the CAN Control Register. MsgVal must also be reset if the messages object is no longer used in operation. For reconfiguration of message objects during normal operation see Section 26.7.6 and Section 26.7.7 .
UMask		Use Acceptance Mask
	0	Mask bits (Msk[28:0], MXtd and MDir) are ignored and not used for acceptance filtering.
	1	Mask bits are used for acceptance filtering. Note: If the UMask bit is set to 1, the message object's mask bits have to be programmed during initialization of the message object before MsgVal is set to 1.
ID[28:0]		Message Identifier
	ID[28:0]	29-bit ("extended") identifier bits
	ID[28:18]	11-bit ("standard") identifier bits
Msk[28:0]		Identifier Mask
	0	The corresponding bit in the message identifier is not used for acceptance filtering (don't care).
	1	The corresponding bit in the message identifier is used for acceptance filtering.
Xtd		Extended Identifier
	0	The 11-bit ("standard") identifier will be used for this message object.
	1	The 29-bit ("extended") identifier will be used for this message object.
MXtd		Mask Extended Identifier
	0	The extended identifier bit (IDE) has no effect on the acceptance filtering.
	1	The extended identifier bit (IDE) is used for acceptance filtering. Note: When 11-bit ("standard") identifiers are used for a message object, the identifiers of received data frames are written into bits ID[28:18]. For acceptance filtering, only these bits together with mask bits Msk[28:18] are considered.
Dir		Message Direction
	0	Direction = receive: On TxRqst, a remote frame with the identifier of this message object is transmitted. On reception of a data frame with matching identifier, the message is stored in this message object.
	1	Direction = transmit: On TxRqst, a data frame is transmitted. On reception of a remote frame with matching identifier, the TxRqst bit of this message object is set (if RmtEn = 1).
MDir		Mask Message Direction
	0	The message direction bit (Dir) has no effect on the acceptance filtering.
	1	The message direction bit (Dir) is used for acceptance filtering.

Table 26-2. Message Object Field Descriptions (continued)

Name	Value	Description
EOB	0	End of Block The message object is part of a FIFO Buffer block and is not the last message object of this FIFO Buffer block.
	1	The message object is a single message object or the last message object in a FIFO Buffer Block. Note: This bit is used to concatenate multiple message objects to build a FIFO Buffer. For single message objects (not belonging to a FIFO Buffer), this bit must always be set to 1.
NewDat	0	New Data No new data has been written into the data bytes of this message object by the Message Handler since the last time when this flag was cleared by the CPU.
	1	The Message Handler or the CPU has written new data into the data bytes of this message object.
MsgLst	0	Message Lost (only valid for message objects with direction = receive) No message was lost since the last time when this bit was reset by the CPU.
	1	The Message Handler stored a new message into this message object when NewDat was still set, so the previous message has been overwritten.
RxIE	0	Receive Interrupt Enable IntPnd will not be triggered after the successful reception of a frame.
	1	IntPnd will be triggered after the successful reception of a frame.
TxIE	0	Transmit Interrupt Enable IntPnd will not be triggered after the successful transmission of a frame.
	1	IntPnd will be triggered after the successful transmission of a frame.
IntPnd	0	Interrupt Pending This message object is not the source of an interrupt.
	1	This message object is the source of an interrupt. The Interrupt Identifier in the Interrupt Register will point to this message object if there is no other interrupt source with higher priority.
RmtEn	0	Remote Enable At the reception of a Remote Frame, TxRqst is not changed.
	1	At the reception of a Remote Frame, TxRqst is set.
TxRqst	0	Transmit Request This message object is not waiting for a transmission.
	1	The transmission of this message object is requested and is not yet done.
DLC[3:0]	0-8	Data Length Code Data Frame has 0-8 data bytes.
	9-15	Data Frame has 8 data bytes. Note: The Data Length Code of a message object must be defined to the same value as in the corresponding objects with the same identifier at other nodes. When the Message Handler stores a data frame, it will write the DLC to the value given by the received message.
Data 0		1st data byte of a CAN Data Frame
Data 1		2nd data byte of a CAN Data Frame
Data 2		3rd data byte of a CAN Data Frame
Data 3		4th data byte of a CAN Data Frame
Data 4		5th data byte of a CAN Data Frame
Data 5		6th data byte of a CAN Data Frame
Data 6		7th data byte of a CAN Data Frame
Data 7		8th data byte of a CAN Data Frame Note: Byte Data 0 is the first data byte shifted into the shift register of the CAN Core during a reception, byte Data 7 is the last. When the Message Handler stores a data frame, it will write all the eight data bytes into a message object. If the Data Length Code is less than 8, the remaining bytes of the message object may be overwritten by undefined values.

26.5.2 Addressing Message Objects in RAM

The starting location of a particular message object in RAM is:

Message RAM base address + (message object number) × 0x20.

This means that Message Object 1 starts at offset 0x0020; Message Object 2 starts at offset 0x0040, and so on.

NOTE: 0 is not a valid message object number. At address 0x0000, message object number 64 is located. Writing to the address of an unimplemented message object may overwrite an implemented message object.

The base address for DCAN1 RAM is FF1E 0000h, for DCAN2 RAM is FF1C 0000h, and for DCAN3 RAM is FF1A 0000h.

Message Object number 1 has the highest priority.

Table 26-3. Message RAM Addressing in Debug/Suspend and RDA Mode

Message Object Number	Base Address Offset	Word Number	Debug/Suspend mode, see Section 26.5.3	RDA mode, see Section 26.5.4
1	0x0020	1	Parity	Data Bytes 4-7
	0x0024	2	MXtd, MDir, Mask	Data Bytes 0-3
	0x0028	3	Xtd, Dir, ID	ID[27:0], DLC
	0x002C	4	Ctrl	Mask, Xtd, Dir, ID[28]
	0x0030	5	Data Bytes 3-0	Parity, Ctrl, MXtd, MDir
	0x0034	6	Data Bytes 7-4	--
:	:	:	:	:
31	0x03E0	1	Parity	Data Bytes 4-7
	0x03E4	2	MXtd, MDir, Mask	Data Bytes 0-3
	0x03E8	3	Xtd, Dir, ID	ID[27:0], DLC
	0x03EC	4	Ctrl	Mask, Xtd, Dir, ID[28]
	0x03F0	5	Data Bytes 3-0	Parity, Ctrl, MXtd, MDir
	0x03F4	6	Data Bytes 7-4	--
:	:	:	:	:
63	0x07E0	1	Parity	Data Bytes 4-7
	0x07E4	2	MXtd, MDir, Mask	Data Bytes 0-3
	0x07E8	3	Xtd, Dir, ID	ID[27:0], DLC
	0x07EC	4	Ctrl	Mask, Xtd, Dir, ID[28]
	0x07F0	5	Data Bytes 3-0	Parity, Ctrl, MXtd, MDir
	0x07F4	6	Data Bytes 7-4	--
64	0x0000	1	Parity	Data Bytes 4-7
	0x0004	2	MXtd, MDir, Mask	Data Bytes 0-3
	0x0008	3	Xtd, Dir, ID	ID[27:0], DLC
	0x000C	4	Ctrl	Mask, Xtd, Dir, ID[28]
	0x0010	5	Data Bytes 3-0	Parity, Ctrl, MXtd, MDir
	0x0014	6	Data Bytes 7-4	--

26.5.3 Message RAM Representation in Debug/Suspend Mode

In Debug/Suspend mode, the Message RAM will be memory mapped. This allows the external debug unit to access the Message RAM.

NOTE: During Debug/Suspend Mode, the Message RAM cannot be accessed via the IFx register sets.

Figure 26-5. Message RAM Representation in Debug/Suspend Mode

Bit	31 15	30 14	29 13	28 12	27 11	26 10	25 9	24 8	23 7	22 6	21 5	20 4	19 3	18 2	17 1	16 0
MsgAddr + 0x00	Reserved															
	Reserved												Parity[4:0]			
MsgAddr + 0x04	MXtd	MDir	Rsvd	Msk[28:16]												
	Msk[15:0]															
MsgAddr + 0x08	Rsvd	Xtd	Dir	ID[28:16]												
	ID[15:0]															
MsgAddr + 0x0C	Reserved															
	Rsvd	MsgLst	Rsvd	UMask	TxE	RxE	RmtEn	Rsvd	EOB	Reserved				DLC[3:0]		
MsgAddr + 0x10	Data 3									Data 2						
	Data 1									Data 0						
MsgAddr + 0x14	Data 7									Data 6						
	Data 5									Data 4						

26.5.4 Message RAM Representation in Direct Access Mode

When the RDA bit in Test Register is set while the DCAN module is in Test Mode (Test bit in CAN control register is set), the CPU has direct access to the Message RAM. Due to the 32-bit bus structure, the RAM is split into word lines to support this feature. The CPU has access to one word line at a time only.

In RAM Direct Access mode, the RAM is represented by a continuous memory space within the address frame of the DCAN module, starting at the Message RAM base address.

NOTE: During Direct Access Mode, the Message RAM cannot be accessed via the IFx register sets. Before entering RDA mode, it must be ensured that the Init bit is set to avoid any conflicts with the message handler accessing the message RAM.

Any read or write to the RAM addresses for RamDirectAccess during normal operation mode (TestMode bit or RDA bit is not set) will be ignored.

Writes to Reserved bits have no effect.

Figure 26-6. Message RAM Representation in RAM Direct Access Mode

Bit	31 15	30 14	29 13	28 12	27 11	26 10	25 9	24 8	23 7	22 6	21 5	20 4	19 3	18 2	17 1	16	
MsgAddr + 0x00	Data 4								Data 5								
	Data 6								Data 7								
MsgAddr + 0x04	Data 0								Data 1								
	Data 2								Data 3								
MsgAddr + 0x08	ID[27:12]																
	ID[11:0]												DLC[3:0]				
MsgAddr + 0x0C	Msk[28:13]																
	Msk[12:0]														Xtd		Dir
MsgAddr + 0x10	Reserved																
	Parity[3:0]				Reserved				MsgLst	UMask	TxE	RxE	RmtEn	EOB	MXtd	MDir	

26.6 Message Interface Register Sets

Accesses to the Message RAM are performed via the Interface Register sets.

1. Interface Register 1 and 2
2. Interface Register 3

The third Interface Register set IF3 can be configured to automatically receive control and user data from the Message RAM when a message object has been updated after reception of a CAN message. The CPU does not need to initiate the transfer from Message RAM to IF3 Register set.

The Message Handler avoids potential conflicts between concurrent accesses to Message RAM and CAN frame reception/transmission.

There are two modes where the Message RAM can be directly accessed by the CPU:

1. In Debug/Suspend mode (see [Section 26.5.3](#))
2. In RAM Direct Access (RDA) mode (see [Section 26.5.4](#))

For the Message RAM Base address, please refer to the device datasheet.

A complete message object (see [Section 26.5.1](#)) or parts of the message object may be transferred between the Message RAM and the IF1/IF2 Register set (see [Section 26.17.20](#)) in one single transfer.

26.6.1 Message Interface Register Sets 1 and 2

The Interface Register sets IF1 and IF2 provide indirect read/write access from the CPU to the Message RAM. The IF1 and IF2 register sets can buffer control and user data to be transferred to and from the message objects.

Table 26-4. Message Interface Register Sets 1 and 2

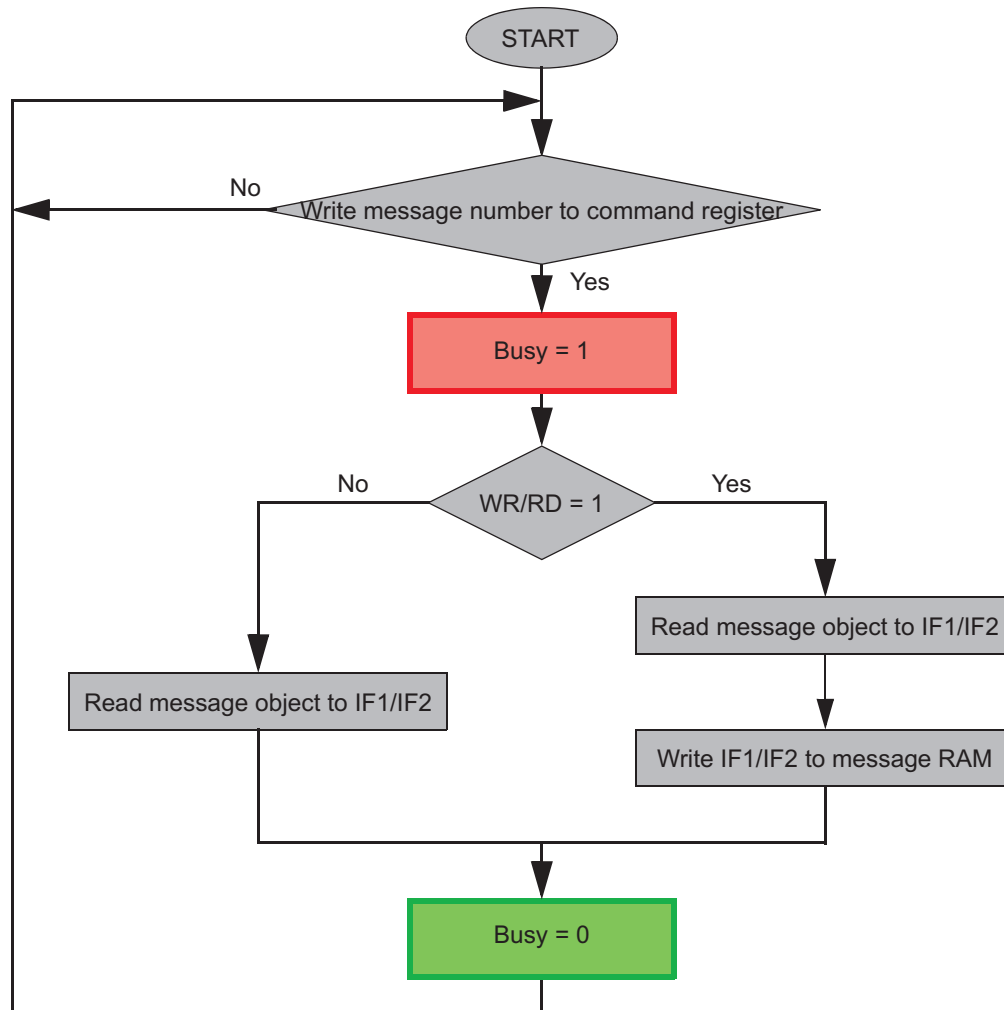
Address [CAN Base +]	IF1 Register Set			Address [CAN Base +]	IF2 Register Set		
	31	16 15	0		31	16 15	0
0x100	IF1 Command Mask	IF1 Command Request		0x120	IF2 Command Mask	IF2 Command Request	
0x104	IF1 Mask 2	IF1 Mask 1		0x124	IF2 Mask 2	IF2 Mask 1	
0x108	IF1 Arbitration 2	IF1 Arbitration 1		0x128	IF2 Arbitration 2	IF2 Arbitration 1	
0x10C	Rsvd	IF1 Message Control		0x12C	Rsvd	IF2 Message Control	
0x110	IF1 Data A 2	IF1 Data A 1		0x130	IF2 Data A 2	IF2 Data A 1	
0x114	IF1 Data B 2	IF1 Data B 1		0x134	IF2 Data B 2	IF2 Data B 1	

26.6.2 Using Message Interface Register Sets 1 and 2

The Command Register addresses the desired message object in the Message RAM and specifies whether a complete message object or only parts should be transferred. The data transfer is initiated by writing the message number to the bits [7:0] of the Command Register.

When the CPU initiates a data transfer between the IF1/IF2 Registers and Message RAM, the Message Handler sets the Busy bit in the respective Command Register to 1. After the transfer has completed, the Busy bit is set back to 0 (see [Figure 26-7](#)).

Figure 26-7. Data Transfer Between IF1 / IF2 Registers and Message RAM



26.6.3 Message Interface Register 3

The IF3 register set can automatically be updated with received message objects without the need to initiate the transfer from Message RAM by CPU. The intention of this feature of IF3 is to provide an interface for the DMA to read packets efficiently.

Table 26-5. Message Interface Register 3

Address [CAN Base +]	IF3 Register Set	
	31	16 15 0
0x140	reserved	IF3 Observation
0x144	IF3 Mask 2	IF3 Mask 1
0x148	IF3 Arbitration 2	IF3 Arbitration 1
0x14C	reserved	IF3 Message Control
0x150	IF3 Data A 2	IF3 Data A 1
0x154	IF3 Data B 2	IF3 Data B 1
...		
0x160	IF3 Update Enable 2	IF3 Update Enable 1
0x164	IF3 Update Enable 4	IF3 Update Enable 3
0x168	IF3 Update Enable 6	IF3 Update Enable 5
0x16C	IF3 Update Enable 8	IF3 Update Enable 7

The automatic update functionality can be programmed for each message object (see IF3 Update Enable Register, [Section 26.17.29](#)).

All valid message objects in Message RAM that are configured for automatic update, will be checked for active NewDat flags. If such a message object is found, it will be transferred to the IF3 register (if no previous DMA transfers are ongoing), controlled by IF3 Observation register. If more than one NewDat flag is active, the message object with the lowest number has the highest priority for automatic IF3 update.

The NewDat bit in the message object will be reset by a transfer to IF3.

If DCAN internal IF3 update is complete, a DMA request is generated. The DMA request stays active until first read access to one of the IF3 registers. The DMA functionality has to be enabled by setting bit DE3 in CAN Control register. Please refer to the device datasheet to find out if this DMA source is available.

NOTE: The IF3 register set can not be used for transferring data into message objects.

26.7 Message Object Configurations

This section describes the possible message object configurations for CAN communication.

26.7.1 Configuration of a Transmit Object for Data Frames

Figure 26-8 shows how a Transmit Object can be initialized.

Figure 26-8. Initialization of a Transmit Object

MsgVal	Arb	Data	Mask	EoB	Dir	NewDat	MsgLst	RxIE	TxIE	IntPnd	RmtEn	TxRqst
1	appl.	appl.	appl.	1	1	0	0	0	appl.	0	appl.	0

The Arbitration bits (ID[28:0] and Xtd bit) are given by the application. They define the identifier and type of the outgoing message. If an 11-bit Identifier (Standard Frame) is used (Xtd = 0), it is programmed to ID[28:18]. In this case, ID[17:0] can be ignored.

The Data Registers (DLC and Data0-7) are given by the application, TxRqst and RmtEn should not be set before the data is valid.

If the TxIE bit is set, the IntPnd bit will be set after a successful transmission of the message object.

If the RmtEn bit is set, a matching received Remote Frame will cause the TxRqst bit to be set; the Remote Frame will autonomously be answered by a Data Frame.

The Mask bits (Msk[28:0], UMask, MXtd, and MDir bits) may be used (UMask = 1) to allow groups of Remote Frames with similar identifiers to set the TxRqst bit. The Dir bit should not be masked. For details, see [Section 26.8.8](#). Identifier masking must be disabled (UMask = 0) if no Remote Frames are allowed to set the TxRqst bit (RmtEn = 0).

26.7.2 Configuration of a Transmit Object for Remote Frames

It is not necessary to configure Transmit Objects for the transmission of Remote Frames. Setting TxRqst for a Receive Object will cause the transmission of a Remote Frame with the same identifier as the Data Frame for which this receive Object is configured.

26.7.3 Configuration of a Single Receive Object for Data Frames

Figure 26-9 shows how a Receive Object for Data Frames can be initialized.

Figure 26-9. Initialization of a Single Receive Object for Data Frames

MsgVal	Arb	Data	Mask	EoB	Dir	NewDat	MsgLst	RxIE	TxIE	IntPnd	RmtEn	TxRqst
1	appl.	appl.	appl.	1	0	0	0	appl.	0	0	0	0

The Arbitration bits (ID[28:0] and Xtd bit) are given by the application. They define the identifier and type of accepted received messages. If an 11-bit Identifier (Standard Frame) is used (Xtd = 0), it is programmed to ID[28:18]. In this case, ID[17:0] can be ignored. When a Data Frame with an 11-bit Identifier is received, ID[17:0] will be set to 0.

The Data Length Code (DLC) is given by the application. When the Message Handler stores a Data Frame in the message object, it will store the received Data Length Code and eight data bytes. If the Data Length Code is less than 8, the remaining bytes of the message object may be overwritten by non specified values.

The Mask bits (Msk[28:0], UMask, MXtd, and MDir bits) may be used (UMask = 1) to allow groups of Data Frames with similar identifiers to be accepted. The Dir bit should not be masked in typical applications. If some bits of the Mask bits are set to “don’t care”, the corresponding bits of the Arbitration Register will be overwritten by the bits of the stored Data Frame.

If the RxIE bit is set, the IntPnd bit will be set when a received Data Frame is accepted and stored in the message object.

If the TxRqst bit is set, the transmission of a Remote Frame with the same identifier as actually stored in the Arbitration bits will be triggered. The content of the Arbitration bits may change if the Mask bits are used (UMask = 1) for acceptance filtering.

26.7.4 Configuration of a Single Receive Object for Remote Frames

Figure 26-10 shows how a Receive Object for Remote Frames can be initialized.

Figure 26-10. Initialization of a Single Receive Object for Remote Frames

MsgVal	Arb	Data	Mask	EoB	Dir	NewDat	MsgLst	RxIE	TxIE	IntPnd	RmtEn	TxRqst
1	appl.	appl.	appl.	1	1	0	0	appl.	0	0	0	0

Receive Objects for Remote Frames may be used to monitor Remote Frames on the CAN bus. The Remote Frame stored in the Receive Object will not trigger the transmission of a Data Frame. Receive Objects for Remote Frames may be expanded to a FIFO buffer, see [Section 26.7.5](#).

UMask must be set to 1. The Mask bits (Msk[28:0], UMask, MXtd, and MDir bits) may be set to “must-match” or to “don’t care”, to allow groups of Remote Frames with similar identifiers to be accepted. The Dir bit should not be masked in typical applications. For details, see [Section 26.8.8](#).

The Arbitration bits (ID[28:0] and Xtd bit) may be given by the application. They define the identifier and type of accepted received Remote Frames. If some bits of the Mask bits are set to “don’t care”, the corresponding bits of the Arbitration bits will be overwritten by the bits of the stored Remote Frame. If an 11-bit Identifier (Standard Frame) is used (Xtd = 0), it is programmed to ID[28:18]. In this case, ID[17:0] can be ignored. When a Remote Frame with an 11-bit Identifier is received, ID[17:0] will be set to 0.

The Data Length Code (DLC) may be given by the application. When the Message Handler stores a Remote Frame in the message object, it will store the received Data Length Code. The data bytes of the message object will remain unchanged.

If the RxIE bit is set, the IntPnd bit will be set when a received Remote Frame is accepted and stored in the message object.

26.7.5 Configuration of a FIFO Buffer

With the exception of the EoB bit, the configuration of Receive Objects belonging to a FIFO Buffer is the same as the configuration of a single Receive Object.

To concatenate multiple message objects to a FIFO Buffer, the identifiers and masks (if used) of these message objects have to be programmed to matching values. Due to the implicit priority of the message objects, the message object with the lowest number will be the first message object of the FIFO Buffer. The EoB bit of all message objects of a FIFO Buffer except the last one have to be programmed to zero. The EoB bits of the last message object of a FIFO Buffer is set to 1, configuring it as the end of the block.

26.7.6 Reconfiguration of Message Objects for the Reception of Frames

A message object with Dir = 0 is configured for the reception of data frames, with Dir = 1 AND Umask = 1 AND RmtEn = 0 it is configured for the reception of remote frames.

It is necessary to reset MsgVal to not valid before changing any of the following configuration and control bits: ID[28:0], Xtd, Dir, DLC, RxIE, TxIE, RmtEn, EoB, Umask, Msk[28:0], MXtd, and MDir.

These parts of a message object may be changed without clearing MsgVal: Data[7:0], TxRqst, NewDat, MsgLst, and IntPnd.

26.7.7 Reconfiguration of Message Objects for the Transmission of Frames

A message object with Dir = 1 AND (Umask = 0 OR RmtEn = 1) is configured for the transmission of data frames.

It is necessary to reset MsgVal to not valid before changing any of the following configuration and control bits: Dir, RxIE, TxIE, RmtEn, EoB, Umask, Msk[28:0], MXtd, and MDir.

These parts of a message object may be changed without clearing MsgVal: ID[28:0], Xtd, DLC, Data[7:0], TxRqst, NewDat, MsgLst, and IntPnd.

26.8 Message Handling

When initialization is finished, the DCAN module synchronizes itself to the traffic on the CAN bus. It does acceptance filtering on received messages and stores those frames that are accepted into the designated message objects.

The application has to update the data of the messages to be transmitted and enable and request their transmission. The transmission is requested automatically when a matching Remote Frame is received.

The application may read messages that are received and accepted. Messages that are not read before the next messages is accepted for the same message object will be overwritten.

Messages may be read based on interrupts or by polling.

26.8.1 Message Handler Overview

The Message Handler state machine controls the data transfer between the Rx/Tx Shift Register of the CAN Core and the Message RAM. It performs the following tasks:

- Data Transfer from Message RAM to CAN Core (messages to be transmitted).
- Data Transfer from CAN Core to the Message RAM (received messages).
- Data Transfer from CAN Core to the Acceptance Filtering unit.
- Scanning of Message RAM for a matching message object (acceptance filtering).
- Scanning the same message object after being changed by IF1/IF2 registers when priority is same or higher as message the object found by last scanning.
- Handling of TxRqst flags.
- Handling of interrupt flags.

The Message Handler registers contains status flags of all message objects grouped into the following topics:

- Transmission Request flags
- New Data flags
- Interrupt Pending Flags
- Message Valid Registers

Instead of collecting the listed status information of each message object via IFx registers separately, these Message Handler registers provides a fast and easy way to get an overview (for example, about all pending transmission requests).

All Message Handler registers are read-only.

26.8.2 Receive/Transmit Priority

The receive/transmit priority for the message objects is attached to the message number, not to the CAN identifier. Message object 1 has the highest priority, while the last implemented message object has the lowest priority. If more than one transmission request is pending, they are serviced due to the priority of the corresponding message object, so messages with the highest priority can be placed in the message objects with the lowest numbers.

The acceptance filtering for received Data Frames or Remote Frames is also done in ascending order of message objects, so a frame that has been accepted by a message object cannot be accepted by another message object with a higher Message Number. The last message object may be configured to accept any Data Frame or Remote Frame that was not accepted by any other message object, for nodes that need to log the complete message traffic on the CAN bus.

26.8.3 Transmission of Messages in Event Driven CAN Communication

If the shift register of the CAN Core is ready for loading and if there is no data transfer between the IFx Registers and Message RAM, the d bits in the Message Valid Register and the TxRqst bits in the Transmission Request Register are evaluated. The valid message object with the highest priority pending transmission request is loaded into the shift register by the Message Handler and the transmission is started. The message object's NewDat bit is reset.

After a successful transmission and if no new data was written to the message object (NewDat = 0) since the start of the transmission, the TxRqst bit will be reset. If TxIE is set, IntPnd will be set after a successful transmission. If the DCAN has lost the arbitration or if an error occurred during the transmission, the message will be retransmitted as soon as the CAN bus is free again. If meanwhile the transmission of a message with higher priority has been requested, the messages will be transmitted in the order of their priority.

If Automatic Retransmission mode is disabled by setting the DAR bit in the CAN Control Register, the behavior of bits TxRqst and NewDat in the Message Control Register of the Interface Register set is as follows:

- When a transmission starts, the TxRqst bit of the respective Interface Register set is reset, while bit NewDat remains set.
- When the transmission has been successfully completed, the NewDat bit is reset.

When a transmission failed (lost arbitration or error) bit NewDat remains set. To restart the transmission, the application has to set TxRqst again.

Received Remote Frames do not require a Receive Object. They will automatically trigger the transmission of a Data Frame, if in the matching Transmit Object the RmtEn bit is set.

26.8.4 Updating a Transmit Object

The CPU may update the data bytes of a Transmit Object any time via the IF1/IF2 Interface Registers, neither d nor TxRqst have to be reset before the update.

Even if only a part of the data bytes are to be updated, all four bytes in the corresponding IF1/IF2 Data A Register or IF1/IF2 Data B Register have to be valid before the content of that register is transferred to the message object. Either the CPU has to write all four bytes into the IF1/IF2 Data Register or the message object is transferred to the IF1/IF2 Data Register before the CPU writes the new data bytes.

When only the data bytes are updated, first 0x87 can be written to bits [23:16] of the Command Register and then the number of the message object is written to bits [7:0] of the Command Register, concurrently updating the data bytes and setting TxRqst with NewDat.

To prevent the reset of TxRqst at the end of a transmission that may already be in progress while the data is updated, NewDat has to be set together with TxRqst in event driven CAN communication. For details, see [Section 26.8.3](#).

When NewDat is set together with TxRqst, NewDat will be reset as soon as the new transmission has started.

26.8.5 Changing a Transmit Object

If the number of implemented message objects is not sufficient to be used as permanent message objects only, the Transmit Objects may be managed dynamically. The CPU can write the whole message (Arbitration, Control, and Data) into the Interface Register. The bits [23:16] of the Command Register can be set to 0xB7 for the transfer of the whole message object content into the message object. Before changing the configuration of a message object, MsgVal has to be reset (see [Section 26.7.7](#)).

If a previously requested transmission of this message object is not completed but already in progress, it will be continued; however it will not be repeated if it is disturbed.

To only update the data bytes of a message to be transmitted, bits [23:16] of the Command Register should be set to 0x87.

NOTE: After the update of the Transmit Object, the Interface Register set will contain a copy of the actual contents of the object, including the part that had not been updated.

26.8.6 Acceptance Filtering of Received Messages

When the arbitration and control bits (Identifier + IDE + RTR + DLC) of an incoming message is completely shifted into the shift register of the CAN Core, the Message Handler starts to scan of the Message RAM for a matching valid message object:

- The Acceptance Filtering unit is loaded with the arbitration bits from the CAN Core shift register.
- Then the arbitration and mask bits (including MsgVal, UMask, NewDat, and EoB) of Message Object 1 are loaded into the Acceptance Filtering unit and are compared with the arbitration bits from the shift register. This is repeated for all following message objects until a matching message object is found, or until the end of the Message RAM is reached.
- If a match occurs, the scanning is stopped and the Message Handler proceeds depending on the type of the frame (Data Frame or Remote Frame) received.

26.8.7 Reception of Data Frames

The Message Handler stores the message from the CAN Core shift register into the respective message object in the Message RAM. Not only the data bytes, but all arbitration bits and the Data Length Code are stored into the corresponding message object. This ensures that the data bytes stay associated to the identifier even if arbitration mask registers are used.

The NewDat bit is set to indicate that new data (not yet seen by the CPU) has been received. The CPU should reset the NewDat bit when it reads the message object. If at the time of the reception the NewDat bit was already set, MsgLst is set to indicate that the previous data (supposedly not seen by the CPU) is lost. If the RxIE bit is set, the IntPnd bit is set, causing the Interrupt Register to point to this message object.

The TxRqst bit of this message object is reset to prevent the transmission of a Remote Frame, while the requested Data Frame has just been received.

26.8.8 Reception of Remote Frames

When a Remote Frame is received, three different configurations of the matching message object have to be considered:

1. Dir = 1 (direction = transmit), RmtEn = 1, UMask = 1 or 0. The TxRqst bit of this message object is set at the reception of a matching Remote Frame. The rest of the message object remains unchanged.
2. Dir = 1 (direction = transmit), RmtEn = 0, UMask = 0. The Remote Frame is ignored, this message object remains unchanged.
3. Dir = 1 (direction = transmit), RmtEn = 0, UMask = 1. The Remote Frame is treated similar to a received Data Frame. At the reception of a matching Remote Frame, the TxRqst bit of this message object is reset. The arbitration and control bits (Identifier + IDE + RTR + DLC) from the shift register are stored in the message object in the Message RAM and the NewDat bit of this message object is set. The data bytes of the message object remain unchanged.

26.8.9 Reading Received Messages

The CPU may read a received message any time via the IFx Interface Registers, the data consistency is guaranteed by the Message Handler state machine.

Typically the CPU will write first 0x7F to bits [23:16] and then the number of the message object to bits [7:0] of the Command Register. That combination will transfer the whole received message from the Message RAM into the Interface Register set. Additionally, the bits NewDat and IntPnd are cleared in the Message RAM (not in the Interface Register set). The values of these bits in the Message Control Register always reflect the status before resetting the bits.

If the message object uses masks for acceptance filtering, the arbitration bits show which of the different matching messages has been received.

The actual value of NewDat shows whether a new message has been received since last time when this message object was read. The actual value of MsgLst shows whether more than one message have been received since the last time when this message object was read. MsgLst will not be automatically reset.

26.8.10 Requesting New Data for a Receive Object

By means of a Remote Frame, the CPU may request another CAN node to provide new data for a receive object. Setting the TxRqst bit of a receive object will cause the transmission of a Remote Frame with the receive object's identifier. This Remote Frame triggers the other CAN node to start the transmission of the matching Data Frame. If the matching Data Frame is received before the Remote Frame could be transmitted, the TxRqst bit is automatically reset.

Setting the TxRqst bit without changing the contents of a message object requires the value 0x84 in bits [23:16] of the Command Register.

26.8.11 Storing Received Messages in FIFO Buffers

Several message objects may be grouped to form one or more FIFO Buffers. Each FIFO Buffer configured to store received messages with a particular (group of) Identifier(s). Arbitration and Mask Registers of the FIFO Buffer's message objects are identical. The EoB (End of Buffer) bits of all but the last of the FIFO Buffer's message objects are 0, in the last one the EoB bit is 1.

Received messages with identifiers matching to a FIFO Buffer are stored into a message object of this FIFO Buffer, starting with the message object with the lowest message number.

When a message is stored into a message object of a FIFO Buffer the NewDat bit of this message object is set. By setting NewDat while EoB is 0, the message object is locked for further write accesses by the Message Handler until the CPU has cleared the NewDat bit.

Messages are stored into a FIFO Buffer until the last message object of this FIFO Buffer is reached. If none of the preceding message objects is released by writing NewDat to 0, all further messages for this FIFO Buffer will be written into the last message object of the FIFO Buffer (EoB = 1) and therefore overwrite previous messages in this message object.

26.8.12 Reading from a FIFO Buffer

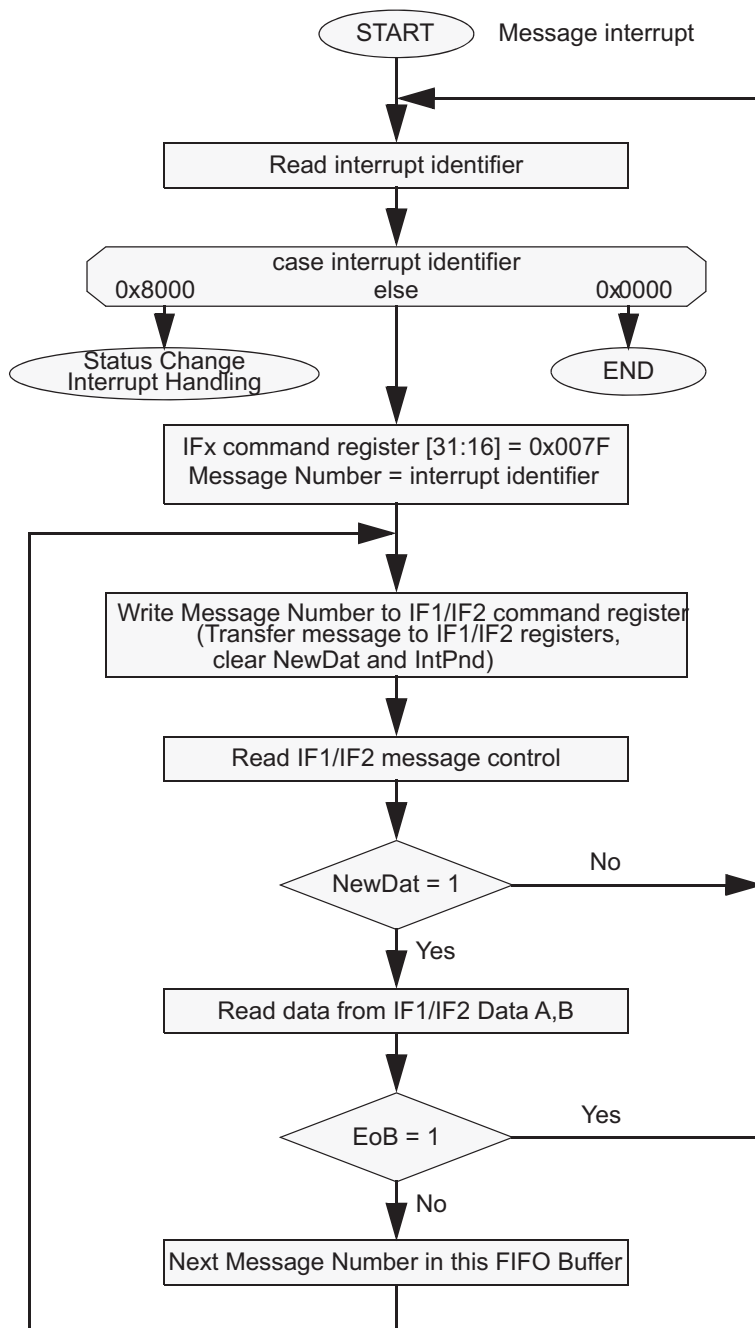
Several messages may be accumulated in a set of message objects that are concatenated to form a FIFO Buffer before the application program is required (in order to avoid the loss of data) to empty the buffer.

A FIFO Buffer of length N will store N-1 plus the last received message since last time it was cleared.

A FIFO Buffer is cleared by reading and resetting the NewDat bits of all its message objects, starting at the FIFO Object with the lowest message number. This should be done in a subroutine following the example shown in [Figure 26-11](#).

NOTE: All message objects of a FIFO buffer needs to be read and cleared before the next batch of messages can be stored. Otherwise true FIFO functionality can not be guaranteed, since the message objects of a partly read buffer will be re-filled according to the normal (descending) priority.

Reading from a FIFO Buffer message object and resetting its NewDat bit is handled the same way as reading from a single message object.

Figure 26-11. CPU Handling of a FIFO Buffer (Interrupt Driven)


26.9 CAN Message Transfer

Once the DCAN is initialized and the Init bit is reset to 0, the CAN Core synchronizes itself to the CAN bus and is ready for message transfer as per the configured message objects.

The CPU may enable the interrupt lines (setting IE0 and IE1 to 1) at the same time when it clears Init and CCE. The status interrupts EIE and SIE may be enabled simultaneously.

The CAN communication can be carried out in any of the following two modes:

1. Interrupt mode
2. Polling mode.

The Interrupt Register points to those message objects with IntPnd = 1. It is updated even if the interrupt lines to the CPU are disabled (IE0 and IE1 are 0).

The CPU may poll all Message Object's NewDat and TxRqst bits in parallel from the NewData X Registers and the Transmission Request X Registers. Polling can be made easier if all Transmit Objects are grouped at the low numbers, all Receive Objects are grouped at the high numbers.

Received messages are stored into their appropriate message objects if they pass acceptance filtering.

The whole message (including all arbitration bits, DLC and up to eight data bytes) is stored into the message object. As a consequence, when the identifier mask is used, the arbitration bits that are masked to "don't care" may change in the message object when a received message is stored.

The CPU may read or write each message at any time via the Interface Registers, as the Message Handler guarantees data consistency in case of concurrent accesses (for reconfiguration, see [Section 26.7.6](#))

If a permanent message object (arbitration and control bits set up during configuration and leaving unchanged for multiple CAN transfers) exists for the message, it is possible to only update the data bytes.

If several transmit messages should be assigned to one message object, the whole message object has to be configured before the transmission of this message is requested.

The transmission of multiple message objects may be requested at the same time. They are subsequently transmitted, according to their internal priority.

Messages may be updated or set to not valid at any time, even if a requested transmission is still pending (for reconfiguration, see [Section 26.7.7](#)). However, the data bytes will be discarded if a message is updated before a pending transmission has started.

Depending on the configuration of the message object, a transmission may be automatically requested by the reception of a remote frame with a matching identifier.

26.9.1 Automatic Retransmission

According to the CAN Specification (ISO11898), the DCAN provides a mechanism to automatically retransmit frames that have lost arbitration or have been disturbed by errors during transmission. The frame transmission service will not be confirmed to the user before the transmission is successfully completed.

By default, this automatic retransmission is enabled. It can be disabled by setting bit DAR (Disable Automatic Retransmission) in CAN Control Register. Further details to this mode are provided in [Section 26.8.3](#).

26.9.2 Auto-Bus-On

Per default, after the DCAN has entered Bus-Off state, the CPU can start a Bus-Off-Recovery sequence by resetting Init bit. If this is not done, the module will stay in Bus-Off state.

The DCAN provides an automatic Auto-Bus-On feature that is enabled by bit ABO in CAN Control Register. If set, the DCAN will automatically start the Bus-Off-Recovery sequence. The sequence can be delayed by a user-defined number of VCLK cycles that can be defined in Auto-Bus-On Time Register.

NOTE: If the DCAN goes Bus-Off due to massive occurrence of CAN bus errors, it stops all bus activities and automatically sets the Init bit. Once the Init bit has been reset by the CPU or due to the Auto-Bus-On feature, the device will wait for 129 occurrences of Bus Idle (equal to 129×11 consecutive recessive bits) before resuming normal operation. At the end of the Bus-Off recovery sequence, the error counters will be reset.

26.10 Interrupt Functionality

Interrupts can be generated on two interrupt lines:

1. DCAN0INT line
2. DCAN1INT line

These lines can be enabled by setting the IE0 and IE1 bits, respectively, in the CAN Control Register.

The DCAN provides three groups of interrupt sources: Message Object Interrupts, Status Change Interrupts and Error Interrupts (see [Figure 26-12](#) and [Figure 26-13](#)).

The source of an interrupt can be determined by the interrupt identifiers Int0ID / Int1ID in the Interrupt Register (see [Section 26.17.5](#)). When no interrupt is pending, the register will hold the value zero.

Each interrupt line remains active until the dedicated field in the Interrupt Register DCAN INT (Int0ID / Int1ID) again reach zero (this means the cause of the interrupt is reset), or until IE0 / IE1 are reset.

The value 0x8000 in the Int0ID field indicates that an interrupt is pending because the CAN Core has updated (not necessarily changed) the Error and Status Register (Error Interrupt or Status Interrupt). This interrupt has the highest priority. The CPU can update (reset) the status bits WakeUpPnd, RxOk, TxOk and LEC by reading the Error and Status Register DCAN ES, but a write access of the CPU will never generate or reset an interrupt.

Values between 1 and the number of the last message object indicates that the source of the interrupt is one of the message objects, Int0ID resp. Int1ID will point to the pending message interrupt with the highest priority. The Message Object 1 has the highest priority, the last message object has the lowest priority.

An interrupt service routine that reads the message that is the source of the interrupt, may read the message and reset the message object's IntPnd at the same time (ClrIntPnd bit in the IF1/IF2 Command Register). When IntPnd is cleared, the Interrupt Register will point to the next message object with a pending interrupt.

26.10.1 Message Object Interrupts

Message Object interrupts are generated by events from the message objects. They are controlled by the flags IntPND, TxIE, and RxIE, that are described in [Section 26.5.1](#).

Message Object interrupts can be routed to either DCAN0INT or DCAN1INT line, controlled by the Interrupt Multiplexer Register (see [Section 26.17.18](#)).

26.10.2 Status Change Interrupts

The events WakeUpPnd, RxOk, TxOk and LEC in Error and Status Register (DCAN ES) belong to the Status Change Interrupts. The Status Change Interrupt group can be enabled by bit in CAN Control Register.

If SIE is set, a Status Change Interrupt will be generated at each CAN frame, independent of bus errors or valid CAN communication, and also independent of the Message RAM configuration.

Status Change interrupts can only be routed to interrupt line DCAN0INT that has to be enabled by setting IE0 in the CAN Control Register.

NOTE: Reading the Error and Status Register will clear the WakeUpPnd flag. If in global power-down mode, the WakeUpPnd flag is cleared by such a read access before the DCAN module has been waken up by the system, the DCAN may re-assert the WakeUpPnd flag, and a second interrupt may occur (additional information can be found in [Section 26.11.2](#)).

26.10.3 Error Interrupts

The events PER, BOff and EWarn (monitored in Error and Status Register, DCAN ES) belong to the Error Interrupts. The Error Interrupt group can be enabled by setting bit EIE in CAN Control Register.

Error interrupts can only be routed to interrupt line DCAN0INT that has to be enabled by setting IE0 in the CAN Control Register.

Figure 26-12. CAN Interrupt Topology 1

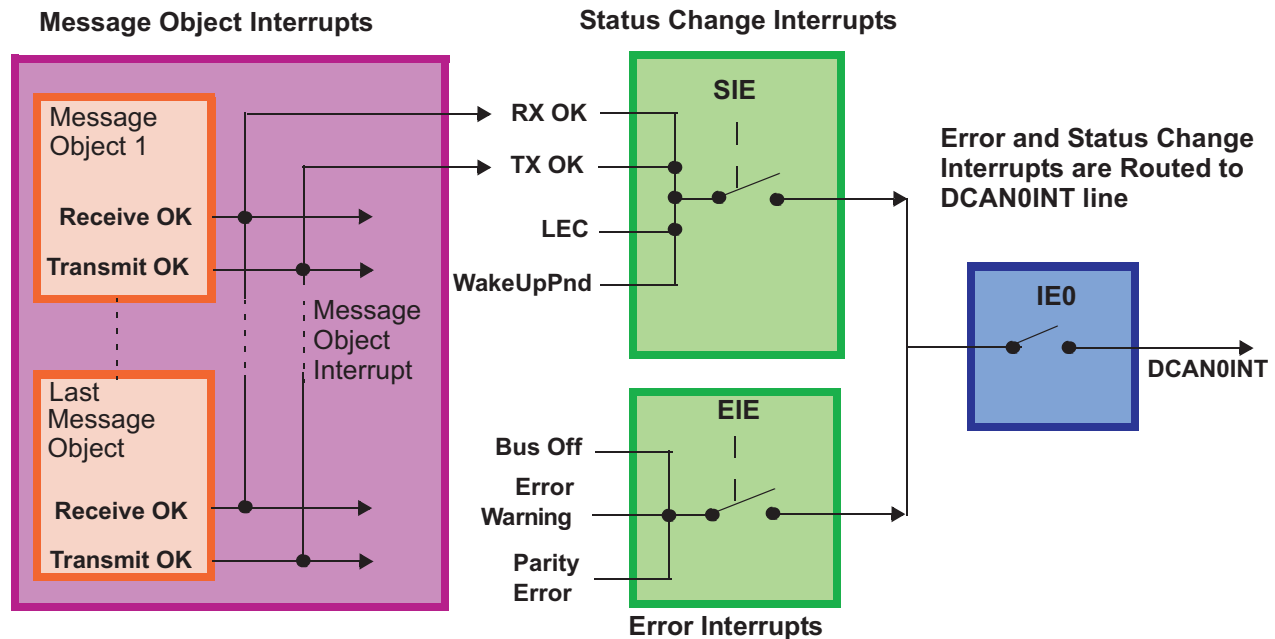
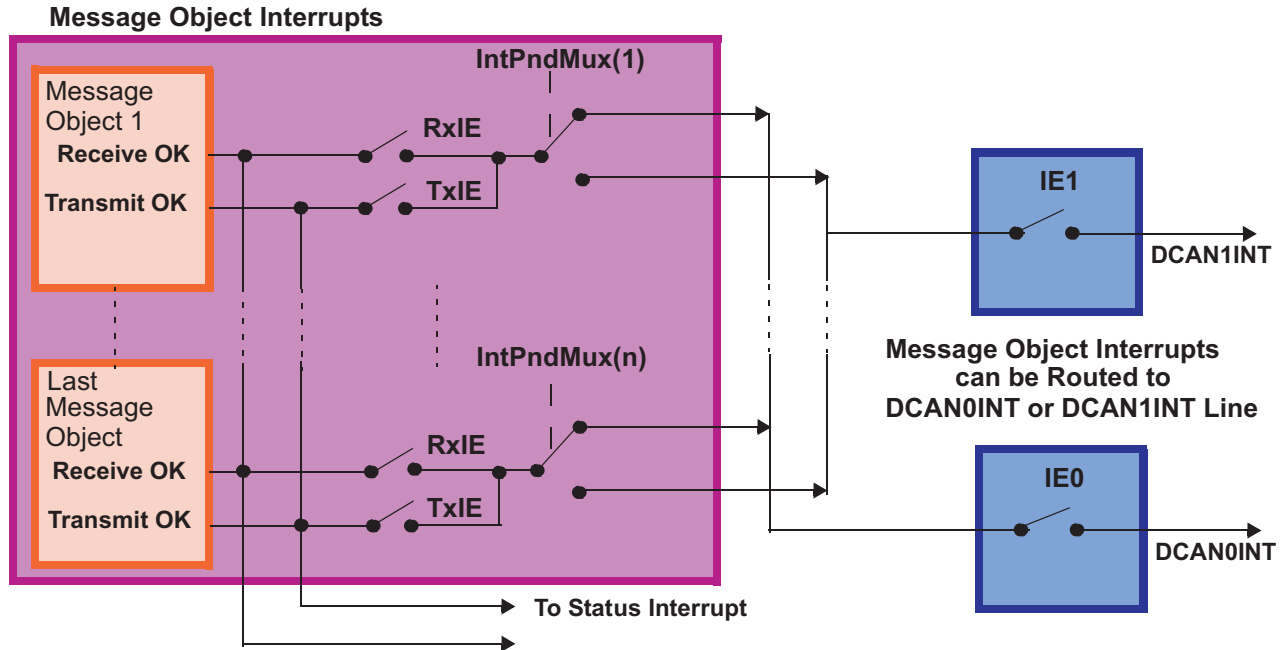


Figure 26-13. CAN Interrupt Topology 2



26.11 Global Power-Down Mode

The device architecture supports a centralized global power down control over the peripheral modules through the Peripheral Central Resource (PCR) module.

26.11.1 Entering Global Power-Down Mode

The global power-down mode for the DCAN is requested by setting the appropriate peripheral power down set bit (PSPWRDWNSETx) in the PCR module.

The DCAN then finishes all transmit requests of the message objects. When all requests are done, the DCAN waits until a bus idle state is recognized. Then it will automatically set the Init bit to indicate that the global power-down mode has been entered.

26.11.2 Wakeup From Global Power-Down Mode

When the DCAN module is in global power-down mode, a CAN bus activity detection circuit exists, which can be active, if enabled. If this circuit is active, on occurrence of a dominant CAN bus level, the DCAN will set the WakeUpPnd bit in Error and Status Register (DCAN ES).

If Status Interrupts are enabled, also an interrupt will be generated. This interrupt could be used by the application to wakeup the DCAN. For this, the application needs to set the appropriate peripheral power down clear bit (PSPWRDWNCLR) in the PCR module, and to clear the Init bit in CAN Control Register.

After the Init bit has been cleared, the DCAN module waits until it detects 11 consecutive recessive bits on the CAN_RX pin and then goes Bus-Active again.

NOTE: The CAN transceiver circuit has to stay active during CAN bus activity detection. The first CAN message, which initiates the bus activity, cannot be received. This means that the first message received in power-down mode is lost.

26.12 Local Power-Down Mode

Besides the centralized power down mechanism controlled by the PCR module (global power down, see [Section 26.15](#)), the DCAN supports a local power-down mode that can be controlled within the DCAN control registers.

26.12.1 Entering Local Power-Down Mode

The local power-down mode is requested by setting the PDR bit in CAN Control Register.

The DCAN then finishes all transmit requests of the message objects. When all requests are done, DCAN waits until a bus idle state is recognized. Then it will automatically set the Init bit in CAN Control Register to prevent any further CAN transfers, and it will also set the PDA bit in CAN Error and Status Register. With setting the PDA bits, the DCAN module indicates that the local power-down mode has been entered.

During local power-down mode, the internal clocks of the DCAN module are turned off, but there is a wake up logic (see [Section 26.12.2](#)) that can be active, if enabled. Also the actual contents of the control registers can be read back.

NOTE: In local low power mode, the application should not clear the Init bit while PDR is set. If there are any messages in the Message RAM configured as to be transmitted and the application resets the init bit, these messages may be sent.

26.12.2 Wakeup From Local Power-Down Mode

There are two ways to wake up the DCAN from local power-down mode:

1. The application could wake up the DCAN module manually by clearing the PDR bit and then clearing the Init bit in CAN Control Register.
2. Alternatively, a CAN bus activity detection circuit can be activated by setting the wake up on bus activity bit (WUBA) in CAN Control Register. If this circuit is active, on occurrence of a dominant CAN bus level, the DCAN will automatically start the wake up sequence. It will clear the PDR bit in CAN Control Register and also clear the PDA bit in Error and Status Register. The WakeUpPnd bit in CAN Error and Status Register will be set. If Status Interrupts are enabled, also an interrupt will be generated. Finally the Init bit in CAN control register will be cleared.

After the Init bit has been cleared, the module waits until it detects 11 consecutive recessive bits on the CAN_RX pin and then goes Bus-Active again.

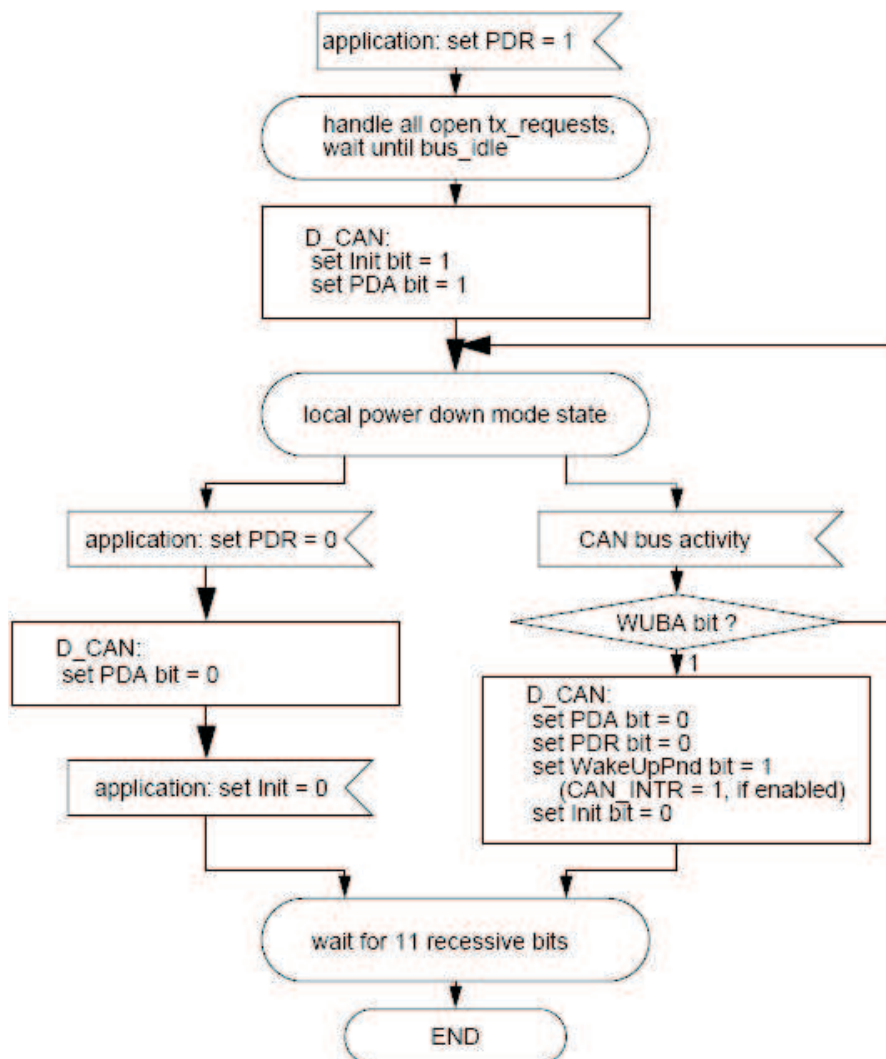
NOTE: The CAN transceiver circuit has to stay active while CAN bus observation. The first CAN message, which initiates the bus activity, cannot be received. This means that the first message received in power-down and automatic wake-up mode, is lost.

[Figure 26-14](#) shows a flow diagram about entering and leaving local power-down mode.

26.13 GIO Support

The CAN_RX and CAN_TX pins of each DCAN module can be used as general purpose IO pins, if CAN functionality is not needed. This function is controlled by the CAN TX IO Control register (see [Section 26.17.30](#)) and the CAN RX IO Control register (see [Section 26.17.31](#)).

Figure 26-14. Local Power-Down Mode Flow Diagram



26.14 Test Modes

The DCAN provides several test modes that are mainly intended for production tests or self test.

For all test modes, the Test bit in the CAN Control Register needs to be set to 1. This enables write access to the Test Register.

NOTE: When using any of the Loop Back modes, it must be ensured by software that all message transfers are finished before setting the Init bit to 1.

26.14.1 Silent Mode

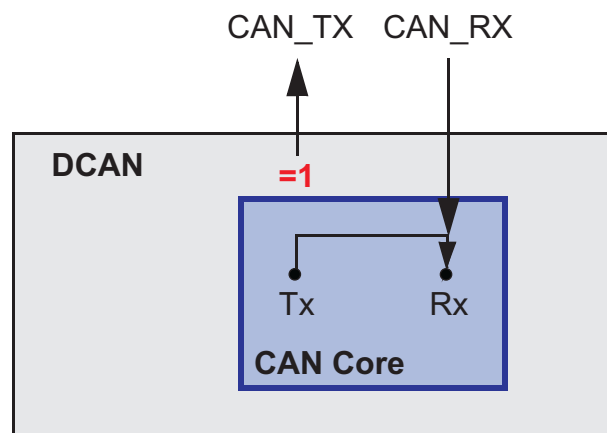
The Silent Mode may be used to analyze the traffic on the CAN bus without affecting it by sending dominant bits (for example, acknowledge bit, overload flag, active error flag). The DCAN is still able to receive valid data frames and valid remote frames, but it will not send any dominant bits. However, these are internally routed to the CAN Core.

Figure 26-15 shows the connection of signals CAN_TX and CAN_RX to the CAN Core in Silent Mode.

Silent Mode can be activated by setting the Silent bit in Test Register to 1.

In ISO 11898-1, the Silent Mode is called the Bus Monitoring Mode.

Figure 26-15. CAN Core in Silent Mode



26.14.2 Loop Back Mode

The Loop Back Mode is mainly intended for hardware self-test functions. In this mode, the CAN Core uses internal feedback from Tx output to Rx input. Transmitted messages are treated as received messages, and can be stored into message objects if they pass acceptance filtering. The actual value of the CAN_RX input pin is disregarded by the CAN Core. Transmitted messages still can be monitored at the CAN_TX pin.

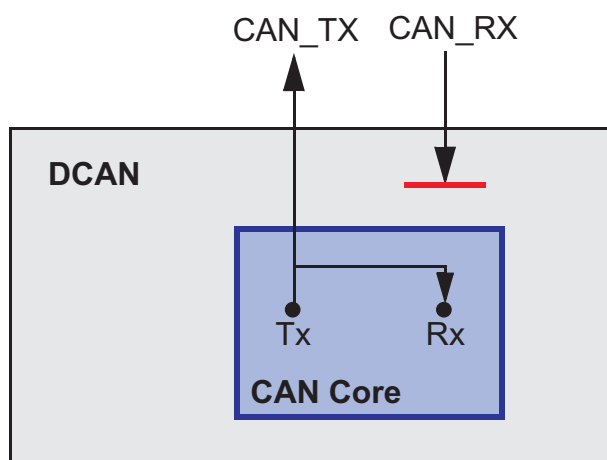
In order to be independent from external stimulation, the CAN Core ignores acknowledge errors (recessive bit sampled in the acknowledge slot of a data/remote frame) in Loop Back Mode.

Figure 26-16 shows the connection of signals CAN_TX and CAN_RX to the CAN Core in Loop Back Mode.

Loop Back Mode can be activated by setting the LBack bit in Test Register to 1.

NOTE: In Loop Back mode, the signal path from CAN Core to Tx pin, the Tx pin itself, and the signal path from Tx pin back to CAN Core are disregarded. For including these into the testing, see External Loop Back mode ([Section 26.14.3](#)).

Figure 26-16. CAN Core in Loop Back Mode



26.14.3 External Loop Back Mode

The External Loop Back Mode is similar to the Loop Back Mode, however it includes the signal path from CAN Core to Tx pin, the Tx pin itself, and the signal path from Tx pin back to CAN Core. When External Loop Back Mode is selected, the input of the CAN core is connected to the input buffer of the Tx pin.

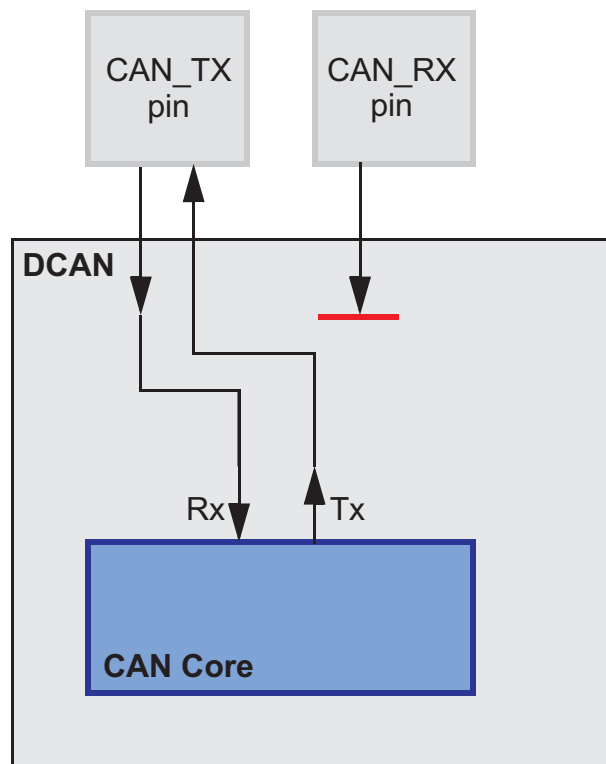
With this configuration, the Tx pin IO circuit can be tested.

External Loop Back Mode can be activated by setting the EXL bit in Test Register to 1.

Figure 26-17 shows the connection of signals CAN_TX and CAN_RX to the CAN Core in External Loop Back Mode.

NOTE: When Loop Back Mode is active (LBack bit is set), the EXL bit will be ignored.

Figure 26-17. CAN Core in External Loop Back Mode

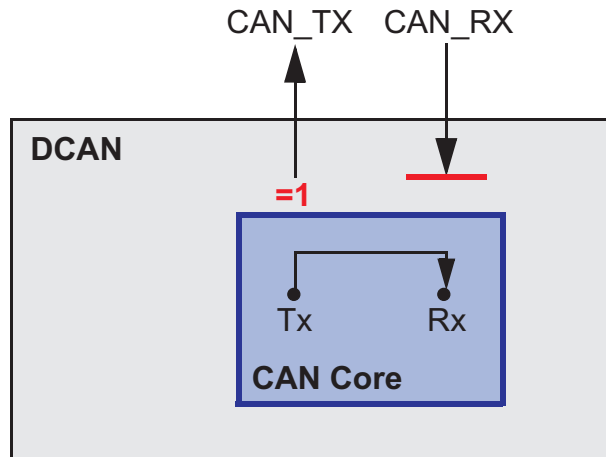


26.14.4 Loop Back Combined with Silent Mode

It is also possible to combine Loop Back Mode and Silent Mode by setting the LBack and Silent bits at the same time. This mode can be used for a “Hot Self-test”, that is, the DCAN hardware can be tested without affecting the CAN network. In this mode, the CAN_RX pin is disconnected from the CAN Core and no dominant bits will be sent on the CAN_TX pin.

Figure 26-18 shows the connection of the signals CAN_TX and CAN_RX to the CAN Core in case of the combination of Loop Back Mode with Silent Mode.

Figure 26-18. CAN Core in Loop Back Combined with Silent Mode



26.14.5 Software Control of CAN_TX Pin

Four output functions are available for the CAN transmit pin CAN_TX. Additionally to its default function (serial data output), the CAN_TX pin can drive constant dominant or recessive values, or it can drive the CAN Sample Point signal to monitor the CAN Core's bit timing.

Combined with the readable value of the CAN_RX pin, this can be used to check the physical layer of the CAN bus.

The output mode of pin CAN_TX is selected by programming the Test Register Tx bits as described in [Section 26.17.6](#).

NOTE: The software control for pin CAN_TX interferes with CAN protocol functions. For CAN message transfer or any of the test modes, Loop Back Mode, External Loop Back Mode, or Silent Mode, the CAN_TX pin should operate in its default functionality.

26.15 Parity Check Mechanism

The DCAN provides a parity check mechanism to ensure data integrity of Message RAM data. For each word (32 bits) in Message RAM, one parity bit will be calculated. The formation of the different words is according to the Message RAM representation in RDA mode (see [Section 26.5.4](#)).

Parity information is stored in the Message RAM on write accesses and will be checked against the stored parity bit from Message RAM on read accesses.

The parity check functionality can be enabled or disabled by PMD bit field in CAN Control Register.

In case of disabled parity check, the parity bits in message RAM will be left unchanged on write access to data area and no check will be done on read access.

If parity checking is enabled, parity bits will be automatically generated and checked by the DCAN. The parity bits could be read in Debug/Suspend mode (see [Section 26.5.3](#)) or in RDA mode (see [Section 26.5.4](#)). However, direct write access to the parity bits is only possible in this two modes with parity check disabled.

A parity bit will be set, if the modulo-2-sum of the data bits is 1. This definition is equivalent to: The parity bit will be set, if the number of 1 bits in the data is odd.

NOTE: Parity scheme can be changed via the System module DEV Parity Control Register1 on device basis for all peri RAMs.

26.15.1 Behavior on Parity Error

On any read access to Message RAM (for example, during start of a CAN frame transmission), the parity of the message object will be checked. If a parity error is detected, the PER bit in Error and Status Register will be set. If error interrupts are enabled, also an interrupt would be generated. In order to avoid the transmission of invalid data over the CAN bus, the d bit of the message object will be reset.

The message object data can be read by the host CPU, independently of parity errors. Thus, the application has to ensure that the read data is valid, for example, by immediately checking the Parity Error Code register on parity error interrupt.

NOTE: During RAM initialization, no parity check will be done.

26.15.2 Parity Testing

Testing the parity mechanism can be done by enabling the bit RDA (RamDirectAccess) and manually writing the parity bits directly to the dedicated RAM locations. With this, data and parity bits could be checked when reading directly from RAM.

NOTE: If parity check was disabled, the application has to ensure correct parity bit handling in order to prevent parity errors later on when parity check is enabled.

26.16 Debug/Suspend Mode

When the CPU is halted during debug, all DCAN registers are visible and can be inspected and modified by the CPU.

In addition, the Message RAM is directly memory-mapped as described in [Table 26-3](#).

The CAN controller provides two options for entering the debug/suspend state. The options are controlled by the IDS bit in the CAN Control Register (DCAN CTL). By default, when IDS is 0, the DCAN controller completes any active transfers on the CAN bus and waits until the bus is idle before halting. When IDS is 1, the DCAN halts immediately as soon as the CPU is halted.

The InitDbg bit in DCAN CTL register indicates when the DCAN controller has actually entered the debug/suspend state.

NOTE: During Debug/Suspend Mode, the Message RAM cannot be accessed via the IFx register sets.

Writing to control registers in debug/suspend mode may influence the CAN state machine and further message handling.

For debug support, the auto clear functionality of the following DCAN registers is disabled:

- Error and Status Register (clear of status flags by read)
- IF1/IF2 Command Registers (clear of DMA Active flag by read-write)

26.17 DCAN Control Registers

[Table 26-6](#) lists the control registers of the DCAN. After hardware reset, the registers of the DCAN hold the values shown in the register descriptions. The base address for the control registers is FFF7 DC00h for DCAN1, FFF7 DE00h for DCAN2, and FFF7 E000h for DCAN3.

Additionally, the Bus-Off state is reset and the CAN_TX pin is set to recessive (HIGH). The Init bit in the CAN Control Register is set to enable the software initialization. The DCAN will not influence the CAN bus until the CPU resets Init to 0.

Table 26-6. DCAN Control Registers

Offset	Acronym	Register Description	Section
00h	DCAN CTL	CAN Control Register	Section 26.17.1
04h	DCAN ES	Error and Status Register	Section 26.17.2
08h	DCAN ERRC	Error Counter Register	Section 26.17.3
0Ch	DCAN BTR	Bit Timing Register	Section 26.17.4
10h	DCAN INT	Interrupt Register	Section 26.17.5
14h	DCAN TEST	Test Register	Section 26.17.6
1Ch	DCAN PERR	Parity Error Code Register	Section 26.17.7
20h	DCAN REL	Core Release Register	Section 26.17.8
80h	DCAN ABOTR	Auto-Bus-On Time Register	Section 26.17.9
84h	DCAN TXRQX	Transmission Request X Register	Section 26.17.10
88h	DCAN TXRQ12	Transmission Request 12 Register	Section 26.17.11
8Ch	DCAN TXRQ34	Transmission Request 34 Register	Section 26.17.11
90h	DCAN TXRQ56	Transmission Request 56 Register	Section 26.17.11
94h	DCAN TXRQ78	Transmission Request 78 Register	Section 26.17.11
98h	DCAN NWDATX	New Data X Register	Section 26.17.12
9Ch	DCAN NWDAT12	New Data 12 Register	Section 26.17.13
A0h	DCAN NWDAT34	New Data 34 Register	Section 26.17.13
A4h	DCAN NWDAT56	New Data 56 Register	Section 26.17.13
A8h	DCAN NWDAT78	New Data 78 Register	Section 26.17.13

Table 26-6. DCAN Control Registers (continued)

Offset	Acronym	Register Description	Section
ACh	DCAN INTPNDX	Interrupt Pending X Register	Section 26.17.14
B0h	DCAN INTPND12	Interrupt Pending 12 Register	Section 26.17.15
B4h	DCAN INTPND34	Interrupt Pending 34 Register	Section 26.17.15
B8h	DCAN INTPND56	Interrupt Pending 56 Register	Section 26.17.15
BCh	DCAN INTPND78	Interrupt Pending 78 Register	Section 26.17.15
C0h	DCAN MSGVALX	Message Valid X Register	Section 26.17.16
C4h	DCAN MSGVAL12	Message Valid 12 Register	Section 26.17.17
C8h	DCAN MSGVAL34	Message Valid 34 Register	Section 26.17.17
CCh	DCAN MSGVAL56	Message Valid 56 Register	Section 26.17.17
D0h	DCAN MSGVAL78	Message Valid 78 Register	Section 26.17.17
D8h	DCAN INTMUX12	Interrupt Multiplexer 12 Register	Section 26.17.18
DCh	DCAN INTMUX34	Interrupt Multiplexer 34 Register	Section 26.17.18
E0h	DCAN INTMUX56	Interrupt Multiplexer 56 Register	Section 26.17.18
E4h	DCAN INTMUX78	Interrupt Multiplexer 78 Register	Section 26.17.18
100h	DCAN IF1CMD	IF1 Command Register	Section 26.17.19
104h	DCAN IF1MSK	IF1 Mask Register	Section 26.17.20
108h	DCAN IF1ARB	IF1 Arbitration Register	Section 26.17.21
10Ch	DCAN IF1MCTL	IF1 Message Control Register	Section 26.17.22
110h	DCAN IF1DATA	IF1 Data A Register	Section 26.17.23
114h	DCAN IF1DATB	IF1 Data B Register	Section 26.17.23
120h	DCAN IF2CMD	IF2 Command Register	Section 26.17.19
124h	DCAN IF2MSK	IF2 Mask Register	Section 26.17.20
128h	DCAN IF2ARB	IF2 Arbitration Register	Section 26.17.21
12Ch	DCAN IF2MCTL	IF2 Message Control Register	Section 26.17.22
130h	DCAN IF2DATA	IF2 Data A Register	Section 26.17.23
134h	DCAN IF2DATB	IF2 Data B Register	Section 26.17.23
140h	DCAN IF3OBS	IF3 Observation Register	Section 26.17.24
144h	DCAN IF3MSK	IF3 Mask Register	Section 26.17.25
148h	DCAN IF3ARB	IF3 Arbitration Register	Section 26.17.26
14Ch	DCAN IF3MCTL	IF3 Message Control Register	Section 26.17.27
150h	DCAN IF3DATA	IF3 Data A Register	Section 26.17.28
154h	DCAN IF3DATB	IF3 Data B Register	Section 26.17.28
160h	DCAN IF3UPD12	IF3 Update Enable 12 Register	Section 26.17.29
164h	DCAN IF3UPD34	IF3 Update Enable 34 Register	Section 26.17.29
168h	DCAN IF3UPD56	IF3 Update Enable 56 Register	Section 26.17.29
16Ch	DCAN IF3UPD78	IF3 Update Enable 78 Register	Section 26.17.29
1E0h	DCAN TIOC	CAN TX IO Control Register	Section 26.17.30
1E4h	DCAN RIOC	CAN RX IO Control Register	Section 26.17.31

26.17.1 CAN Control Register (DCAN CTL)

Figure 26-19. CAN Control Register (DCAN CTL) [offset = 00]

31				26			25		24						
Reserved							WUBA		PDR						
R-0							R/W-0		R/W-0						
23		21		20		19		18		17		16			
Reserved				DE3		DE2		DE1		IE1		InitDbg			
R-0				R/W-0		R/W-0		R/W-0		R/W-0		R-0			
15		14		13		10				9		8			
SWR	Reserved	PMD							ABO		IDS				
R/WP-0		R-0		R/W-5h						R/W-0		R/W-0			
7		6		5		4		3		2		1		0	
Test	CCE	DAR	Reserved		EIE		SIE		IE0		Init				
R/W-0		R/W-0		R/W-0		R-0		R/W-0		R/W-0		R/W-0		R/W-1	

LEGEND: R/W = Read/Write; R = Read only; WP = Write protected by init bit; -n = value after reset

Table 26-7. CAN Control Register Field Descriptions

Bit	Field	Value	Description
31-26	Reserved	0	These bits are always read as 0. Writes have no effect.
25	WUBA	0 1	Automatic wake up on bus activity when in local power-down mode. No detection of a dominant CAN bus level while in local power-down mode. Detection of a dominant CAN bus level while in local power-down mode is enabled. On occurrence of a dominant CAN bus level, the wake up sequence is started. (Additional information can be found in Section 26.12 .) Note: The CAN message, which Initiates the bus activity, cannot be received. This means that the first message received in power-down and automatic wake-up mode, will be lost.
24	PDR	0 1	Request for local low power-down mode. No application request for local low power-down mode. If the application has cleared this bit while DCAN in local power-down mode, also the Init bit has to be cleared. Local power-down mode has been requested by application. The DCAN will acknowledge the local power-down mode by setting bit PDA in Error and Status Register. The local clocks will be turned off by DCAN internal logic (Additional information can be found in Section 26.12).
23-21	Reserved	0	These bits are always read as 0. Writes have no effect.
20	DE3	0 1	Enable DMA request line for IF3. Disabled Enabled Note: A pending DMA request for IF3 remains active until first access to one of the IF3 registers.
19	DE2	0 1	Enable DMA request line for IF2. Disabled Enabled Note: A pending DMA request for IF2 remains active until first access to one of the IF2 registers.
18	DE1	0 1	Enable DMA request line for IF1. Disabled Enabled Note: A pending DMA request for IF1 remains active until first access to one of the IF1 registers.
17	IE1	0 1	Interrupt line 1 enable. Disabled. Module Interrupt DCAN1INT is always low. Enabled. Interrupts will assert line DCAN1INT to 1; line remains active until pending interrupts are processed.

Table 26-7. CAN Control Register Field Descriptions (continued)

Bit	Field	Value	Description
16	InitDbg	0	Internal Init state while debug access. Not in debug mode, or debug mode requested but is not entered.
		1	Debug mode requested and is internally entered; the DCAN is ready for debug accesses.
15	SWR	0	SW Reset enable. Normal operation.
		1	Module is forced to reset state. This bit will automatically get cleared after execution of SW reset after one VBUSP clock cycle. Note: To execute SW reset the following procedure is necessary: 1. Set Init bit to shut down CAN communication. 2. Set SWR bit additionally to Init bit.
14	Reserved	0	This bit is always read as 0. Writes have no effect.
13-10	PMD	5h	Parity enable. Parity function is disabled.
		Others	Parity function is enabled.
9	ABO	0	Auto-Bus-On enable. The Auto-Bus-On feature is disabled.
		1	The Auto-Bus-On feature is enabled.
8	IDS	0	Interruption Debug Support enable. When Debug/Suspend mode is requested, DCAN will wait for a started transmission or reception to be completed before entering Debug/Suspend mode.
		1	When Debug/Suspend mode is requested, DCAN will interrupt any transmission or reception, and enter Debug/Suspend mode immediately.
7	Test	0	Test Mode enable. Normal operation
		1	Test mode
6	CCE	0	Configuration Change Enable. The CPU has no write access to the BTR config register.
		1	The CPU has write access to the BTR config register (when Init bit is set).
5	DAR	0	Disable Automatic Retransmission. Automatic Retransmission of not successful messages is enabled.
		1	Automatic Retransmission is disabled.
4	Reserved	0	This bit is always read as 0. Writes have no effect.
3	EIE	0	Error Interrupt Enable. Disabled. PER, BOff, and EWarn bits cannot generate an interrupt.
		1	Enabled. PER, BOff, and EWarn bits can generate an interrupt at DCAN0INT line and affect the Interrupt Register.
2	SIE	0	Status Change Interrupt Enable. Disabled. WakeUpPnd, RxOk, TxOk, and LEC bits cannot generate an interrupt.
		1	Enabled. WakeUpPnd, RxOk, TxOk, and LEC can generate an interrupt at DCAN0INT line and affect the Interrupt Register.
1	IE0	0	Interrupt line 0 enable. Disabled. Module Interrupt DCAN0INT is always low.
		1	Enabled. Interrupts will assert line DCAN0INT to 1; line remains active until pending interrupts are processed.
0	Init	0	Initialization. Normal operation.
		1	Initialization mode is entered.

NOTE: The Bus-Off recovery sequence (see CAN specification) cannot be shortened by setting or resetting Init bit. If the module goes Bus-Off, it will automatically set the Init bit and stop all bus activities.

When the Init bit is cleared by the application again, the module will then wait for 129 occurrences of Bus Idle (129 × 11 consecutive recessive bits) before resuming normal operation. At the end of the Bus-Off recovery sequence, the error counters will be reset.

After the Init bit is reset, each time when a sequence of 11 recessive bits is monitored, a Bit0 error code is written to the Error and Status Register, enabling the CPU to check whether the CAN bus is stuck at dominant or continuously disturbed, and to monitor the proceeding of the Bus-Off recovery sequence.

26.17.2 Error and Status Register (DCAN ES)

Figure 26-20. Error and Status Register (DCAN ES) [offset = 04h]

31																16																									
Reserved																																									
R-0																																									
15															11										10					9					8						
Reserved															PDA										WakeUpPnd					PER											
R-0															R-0										R/C-0					R/C-0											
7					6					5					4					3					2					0											
BOff					EWarn					EPass					RxOK					TxOK					LEC																
R-0					R-0					R-0					R/C-0					R/C-0					R/S-7h																

LEGEND: R = Read only; C = Clear; S = Set; -n = value after reset

Table 26-8. Error and Status Register Field Descriptions

Bit	Field	Value	Description
31-11	Reserved	0	These bits are always read as 0. Writes have no effect.
10	PDA	0	Local power-down mode acknowledge
		1	DCAN is not in local power-down mode.
9	WakeUpPnd	0	Application request for setting DCAN to local power-down mode was successful. DCAN is in local power-down mode.
		1	Wake Up Pending
8	PER	0	This bit can be used by the CPU to identify the DCAN as the source to wake up the system.
		1	No Wake Up is requested by DCAN.
7	BOff	0	DCAN has initiated a wake up of the system due to dominant CAN bus while module power down. This bit will be reset if Error and Status Register is read.
		1	Parity Error Detected
6	EWarn	0	No parity error has been detected since last read access.
		1	The parity check mechanism has detected a parity error in the Message RAM. This bit will be reset if Error and Status Register is read.
5	EPass	0	Bus-Off State
		1	The CAN module is not Bus-Off state.
4	RxOK	0	The CAN module is in Bus-Off state.
		1	Warning State
3	TxOK	0	Both error counters are below the error warning limit of 96.
		1	At least one of the error counters has reached the error warning limit of 96.

Table 26-8. Error and Status Register Field Descriptions (continued)

Bit	Field	Value	Description
5	EPass	0	Error Passive State On CAN Bus error, the DCAN could send active error frames.
		1	The CAN Core is in the error passive state as defined in the CAN Specification.
4	RxOK	0	Received a message successfully No message has been successfully received since the last time when this bit was read by the CPU. This bit is never reset by DCAN internal events.
		1	A message has been successfully received since the last time when this bit was reset by a read access of the CPU (independent of the result of acceptance filtering). This bit will be reset if Error and Status Register is read.
3	TxOK	0	Transmitted a message successfully No message has been successfully transmitted since the last time when this bit was read by the CPU. This bit is never reset by DCAN internal events.
		1	A message has been successfully transmitted (error free and acknowledged by at least one other node) since the last time when this bit was reset by a read access of the CPU. This bit will be reset if Error and Status Register is read.
2-0	LEC		Last Error Code The LEC field indicates the type of the last error on the CAN bus. This field will be cleared to 0 when a message has been transferred (reception or transmission) without error.
		0	No Error
		1h	Stuff Error: More than five equal bits in a row have been detected in a part of a received message where this is not allowed.
		2h	Form Error: A fixed format part of a received frame has the wrong format.
		3h	Ack Error: The message this CAN Core transmitted was not acknowledged by another node.
		4h	Bit1 Error: During the transmission of a message (with the exception of the arbitration field), the device wanted to send a recessive level (bit of logical value 1), but the monitored bus value was dominant.
		5h	Bit0 Error: During the transmission of a message (or acknowledge bit, or active error flag, or overload flag), the device wanted to send a dominant level (logical value 0), but the monitored bus level was recessive. During Bus-Off recovery, this status is set each time a sequence of 11 recessive bits has been monitored. This enables the CPU to monitor the proceeding of the Bus-Off recovery sequence (indicating the bus is not stuck at dominant or continuously disturbed).
		6h	CRC Error: In a received message, the CRC check sum was incorrect. (CRC received for an incoming message does not match the calculated CRC for the received data).
		7h	No CAN bus event was detected since the last time when CPU has read the Error and Status Register. Any read access to the Error and Status Register re-Initializes the LEC to value 7h.

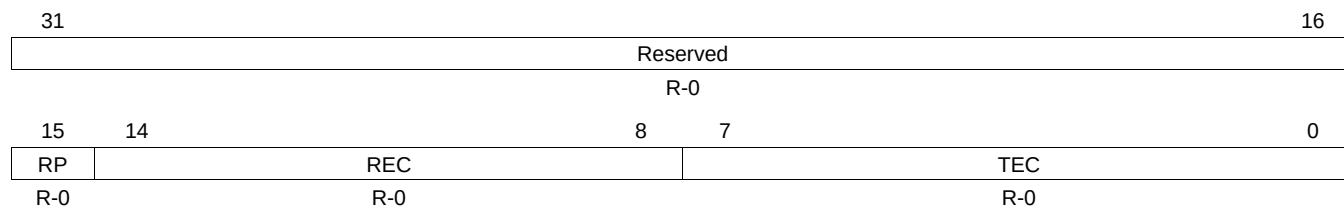
Interrupts are generated by bits PER, BOff, and EWarn (if EIE bit in CAN Control Register is set) and by bits WakeUpPnd, RxOk, TxOk, and LEC (if SIE bit in CAN Control Register is set). A change of bit EPass will not generate an Interrupt.

NOTE: Reading the Error and Status Register clears the WakeUpPnd, PER, RxOk, and TxOk bits and sets the LEC to value 7h. Additionally, the Status Interrupt value (8000h) in the Interrupt Register will be replaced by the next lower priority interrupt value.

For debug support, the auto clear functionality of Error and Status Register (clear of status flags by read) is disabled when in Debug/Suspend mode.

26.17.3 Error Counter Register (DCAN ERRC)

Figure 26-21. Error Counter Register (DCAN ERRC) [offset = 08h]



LEGEND: R = Read only; -n = value after reset

Table 26-9. Error Counter Register Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	These bits are always read as 0. Writes have no effect.
15	RP	0 1	Receive Error Passive The Receive Error Counter is below the error passive level. The Receive Error Counter has reached the error passive level as defined in the CAN Specification.
14-8	REC	0-7Fh	Receive Error Counter. Actual state of the Receive Error Counter. (Values from 0 to 127.)
7-0	TEC	0-FFh	Transmit Error Counter. Actual state of the Transmit Error Counter. (Values from 0 to 255.)

26.17.4 Bit Timing Register (DCAN BTR)

Figure 26-22. Bit Timing Register (DCAN BTR) [offset = 0Ch]



LEGEND: R/W = Read/Write; R = Read only; WP = Write Protected by CCE bit; -n = value after reset

Table 26-10. Bit Timing Register Field Descriptions

Bit	Field	Value	Description
31-20	Reserved	0	These bits are always read as 0. Writes have no effect.
	BRPE	0-Fh	Baud Rate Prescaler Extension. Valid programmed values are 0 to 15. By programming BRPE the Baud Rate Prescaler can be extended to values up to 1024.
15	Reserved	0	This bit is always read as 0. Writes have no effect.
14-12	TSeg2	0-7h	Time segment after the sample point Valid programmed values are 0 to 7. The actual TSeg2 value that is interpreted for the Bit Timing will be the programmed TSeg2 value + 1.
11-8	TSeg1	1h-Fh	Time segment before the sample point Valid programmed values are 1 to 15. The actual TSeg1 value interpreted for the Bit Timing will be the programmed TSeg1 value + 1.
7-6	SJW	0-3h	Synchronization Jump Width Valid programmed values are 0 to 3.The actual SJW value interpreted for the Synchronization will be the programmed SJW value + 1.
5-0	BRP	0-3Fh	Baud Rate Prescaler Value by which the CAN_CLK frequency is divided for generating the bit time quanta. The bit time is built up from a multiple of this quanta. Valid programmed values are 0 to 63.The actual BRP value interpreted for the Bit Timing will be the programmed BRP value + 1.

NOTE: This register is only writable if CCE and Init bits in the CAN Control Register are set.

The CAN bit time may be programmed in the range of 8 to 25 time quanta.

The CAN time quantum may be programmed in the range of 1 to 1024 CAN_CLK periods.

With a CAN_CLK of 8 MHz and BRPE = 00, the reset value of 2301h configures the DCAN for a bit rate of 500kBit/s.

For details, see [Section 26.3.2.1](#).

26.17.5 Interrupt Register (DCAN INT)

Figure 26-23. Interrupt Register (DCAN INT) [offset = 10h]

31		24	23		16
Reserved			Int1ID		
R-0			R-0		
15					0
Int0ID					
R-0					

LEGEND: R = Read only; -n = value after reset

Table 26-11. Interrupt Register Field Descriptions

Bit	Field	Value	Description
31-24	Reserved	0	These bits are always read as 0. Writes have no effect.
23-16	Int1ID	0 1h-40h 41h-FFh	Interrupt 1 Identifier (indicates the message object with the highest pending interrupt). No interrupt is pending. Number of message object that caused the interrupt. Reserved If several interrupts are pending, the CAN Interrupt Register will point to the pending interrupt with the highest priority. The DCAN1INT interrupt line remains active until Int1ID reaches value 0 (the cause of the interrupt is reset) or until IE1 is cleared. A message interrupt is cleared by clearing the message object's IntPnd bit. Among the message interrupts, the message object's interrupt priority decreases with increasing message number.
15-0	Int0ID	0 1h-40h 41h-7FFFh 8000h 8001h-FFFFh	Interrupt 0 Identifier (the number here indicates the source of the interrupt). No interrupt is pending. Number of message object that caused the interrupt. Reserved Error and Status Register value is not 0x07. Reserved If several interrupts are pending, the CAN Interrupt Register will point to the pending interrupt with the highest priority. The DCAN0INT interrupt line remains active until Int0ID reaches value 0 (the cause of the interrupt is reset) or until IE0 is cleared. The Status Interrupt has the highest priority. Among the message interrupts, the message object's interrupt priority decreases with increasing message number.

26.17.6 Test Register (DCAN TEST)

Figure 26-24. Test Register (DCAN TEST) [offset = 14h]

31	Reserved																16
R-0																	
15	Reserved										10	9	8				
R-0											R/WP-0		R/WP-0				
7	6	5	4	3	2	0											
Rx	Tx		LBack		Silent		Reserved										
R-U	R/WP-0		R/WP-0		R/WP-0		R-0										

LEGEND: R/W = Read/Write; R = Read only; WP = Write Protected by Test bit; -n = value after reset; U = Undefined

Table 26-12. Test Register Field Descriptions

Bit	Field	Value	Description
31-10	Reserved	0	These bits are always read as 0. Writes have no effect.
9	RDA	0 1	RAM Direct Access Enable Normal Operation Direct access to the RAM is enabled while in Test Mode.
8	EXL	0 1	External Loop Back Mode Disabled Enabled
7	Rx	0 1	Receive Pin. Monitors the actual value of the CAN_RX pin. The CAN bus is dominant. The CAN bus is recessive.
6-5	Tx	0 1h 2h 3h	Control of CAN_TX pin Normal operation, CAN_TX is controlled by the CAN Core. Sample Point can be monitored at CAN_TX pin. CAN_TX pin drives a dominant value. CAN_TX pin drives a recessive value.
4	LBack	0 1	Loop Back Mode Disabled Enabled
3	Silent	0 1	Silent Mode Disabled Enabled
2-0	Reserved	0	These bits are always read as 0. Writes have no effect.

For all test modes, the Test bit in CAN Control Register needs to be set to 1. If Test bit is set, the RDA, EXL, Tx1, Tx0, LBack, and Silent bits are writable. Bit Rx monitors the state of pin CAN_RX and therefore is only readable. All Test Register functions are disabled when the Test bit is cleared to 0.

NOTE: The Test Register is only writable if Test bit in CAN Control Register is set.

Setting Tx to other than 00 will disturb message transfer.

When the internal loop back mode is active (LBack bit is set), EXL bit will be ignored.

26.17.7 Parity Error Code Register (DCAN PERR)

Figure 26-25. Parity Error Code Register (DCAN PERR) [offset = 1Ch]

31	Reserved															16
R-0																
15	Reserved				11	10	8	7	Message Number							0
R-0					R-U				R-U							

LEGEND: R = Read only; -n = value after reset; U = Undefined

Table 26-13. Parity Error Code Register Field Descriptions

Bit	Field	Value	Description
31-11	Reserved	0	These bits are always read as 0. Writes have no effect.
10-8	Word Number	1h-5h	Word number where parity error has been detected. RDA word number (1 to 5) of the message object (according to the Message RAM representation in RDA mode, see Section 26.5.4).
7-0	Message Number	1h-FFh	Message object number where parity error has been detected. Only values 1h-40h are valid. Values 41h-FFh are invalid.

If a parity error is detected, the PER flag will be set in the Error and Status Register. This bit is not reset by the parity check mechanism; it must be reset by reading the Error and Status Register.

In addition to the PER flag, the Parity Error Code Register will indicate the memory area where the parity error has been detected (message number and word number).

If more than one word with a parity error was detected, the highest word number with a parity error will be displayed.

After a parity error has been detected, the register will hold the last error code until power is removed.

26.17.8 Core Release Register (DCAN REL)

Figure 26-26. Core Release Register (DCAN REL) [offset = 20h]

31	28	27	24	23	20	19	16
REL				STEP			
R-Ah				R-3h			
15	MON				8	7	0
R-5h				DAY			
				R-4h			

LEGEND: R = Read only; -n = value after reset

Table 26-14. Core Release Register (DCAN REL) Field Descriptions

Bit	Field	Value	Description
31-28	REL	0-9h	Core Release. One digit, BCD-coded.
27-24	STEP	0-9h	Step of Core Release. One digit, BCD-coded.
23-20	SUBSTEP	0-9h	Substep of Core Release. One digit, BCD-coded.
19-16	YEAR	0-9h	Design Time Stamp, Year. One digit, BCD-coded. This field is set by constant parameter on DCAN synthesis.
15-8	MON	0-12h	Design Time Stamp, Month. Two digits, BCD-coded. This field is set by constant parameter on DCAN synthesis.
7-0	DAY	0-31h	Design Time Stamp, Day. Two digits, BCD-coded. This field is set by constant parameter on DCAN synthesis.

26.17.9 Auto-Bus-On Time Register (DCAN ABOTR)

Figure 26-27. Auto-Bus-On Time Register (DCAN ABOTR) [offset = 80h]

31	16
ABO_TIME	
R/W-0	
15	0
ABO_TIME	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Table 26-15. Auto-Bus-On Time Register Field Descriptions

Bit	Field	Value	Description
31-0	ABO_TIME	0-FFFF FFFFh	Number of VBUS clock cycles before a Bus-Off recovery sequence is started by clearing the Init bit. This function has to be enabled by setting the ABO bit in the CAN Control Register. The Auto-Bus-On timer is realized by a 32-bit counter that starts to count down to 0 when the module goes Bus-Off. The counter will be reloaded with the preload value of the ABO_TIME register after this phase.

NOTE: On write access to the CAN Control register while Auto-Bus-On timer is running, the Auto-Bus-On procedure will be aborted.

During Debug/Suspend mode, running Auto-Bus-On timer will be paused.

26.17.10 Transmission Request X Register (DCAN TXRQ X)

With the Transmission Request X Register, the CPU can detect if one or more bits in the different Transmission Request Registers are set. Each register bit represents a group of eight message objects. If at least one of the TxRqst bits of these message objects are set, the corresponding bit in the Transmission Request X Register will be set.

Figure 26-28. Transmission Request X Register (DCAN TXRQ X) [offset = 84h]

31															16				
Reserved																			
R-0																			
15		14	13		12	11		10	9	8	7	6	5	4	3	2	1	0	
TxRqstReg8			TxRqstReg7			TxRqstReg6			TxRqstReg5			TxRqstReg4		TxRqstReg3		TxRqstReg2		TxRqstReg1	
R-0			R-0			R-0			R-0			R-0		R-0		R-0		R-0	

LEGEND: R = Read only; -n = value after reset

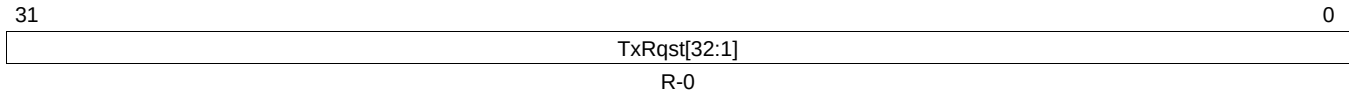
Example

Bit 0 of the Transmission Request X Register represents byte 0 of the Transmission Request 1 Register (DCAN TXRQ12). If one or more bits in this byte are set, bit 0 of the Transmission Request X Register will be set.

26.17.11 Transmission Request Registers (DCAN TXRQ12 to DCAN TXRQ78)

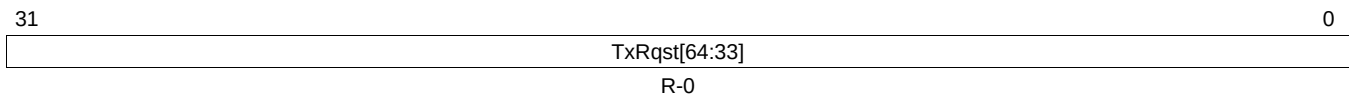
These registers hold the TxRqst bits of the implemented message objects. By reading out these bits, the CPU can check for pending transmission requests. The TxRqst bit in a specific message object can be set/reset by the CPU via the IF1/IF2 Message Interface Registers, or by the Message Handler after reception of a remote frame or after a successful transmission.

Figure 26-29. Transmission Request 12 Register [offset = 88h]



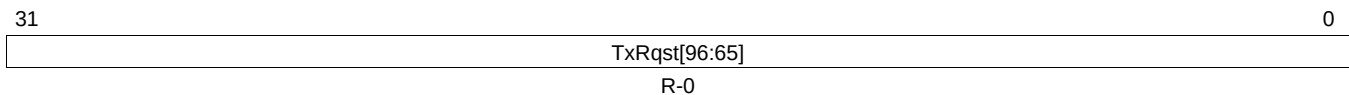
LEGEND: R = Read only; -n = value after reset

Figure 26-30. Transmission Request 34 Register [offset = 8Ch]



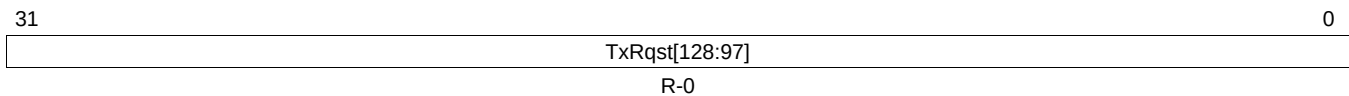
LEGEND: R = Read only; -n = value after reset

Figure 26-31. Transmission Request 56 Register [offset = 90h]



LEGEND: R = Read only; -n = value after reset

Figure 26-32. Transmission Request 78 Register [offset = 94h]



LEGEND: R = Read only; -n = value after reset

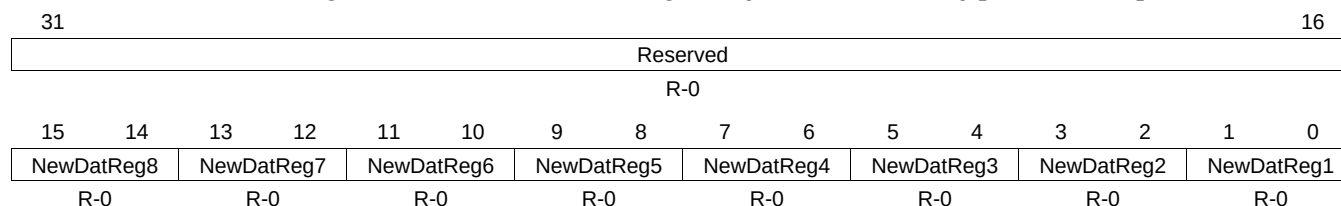
Table 26-16. Transmission Request Registers Field Descriptions

Bit	Name	Value	Description
31-0	TxRqst[n]		Transmission Request Bits (for all message objects)
		0	No transmission has been requested for this message object.
		1	The transmission of this message object is requested and is not yet done.

26.17.12 New Data X Register (DCAN NWDAT X)

With the New Data X Register, the CPU can detect if one or more bits in the different New Data Registers are set. Each register bit represents a group of eight message objects. If at least one of the NewDat bits of these message objects are set, the corresponding bit in the New Data X Register will be set.

Figure 26-33. New Data X Register (DCAN NWDAT X) [offset = 98h]



LEGEND: R = Read only; -n = value after reset

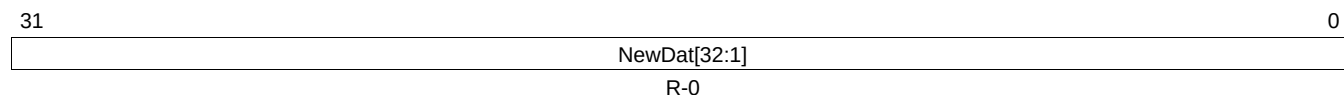
Example

Bit 0 of the New Data X Register represents byte 0 of the New Data 1 Register (DCAN NWDAT12). If one or more bits in this byte are set, bit 0 of the New Data X Register will be set.

26.17.13 New Data Registers (DCAN NWDAT12 to DCAN NWDAT78)

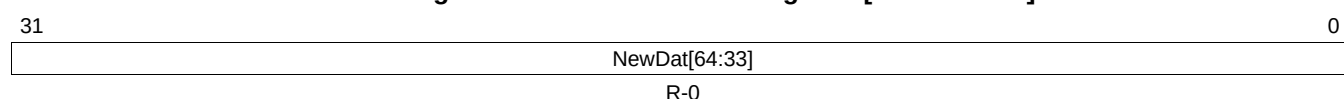
These registers hold the NewDat bits of the implemented message objects. By reading out these bits, the CPU can check for new data in the message objects. The NewDat bit of a specific message object can be set/reset by the CPU via the IF1/IF2 Interface Register sets, or by the Message Handler after reception of a data frame or after a successful transmission.

Figure 26-34. New Data 12 Register [offset = 9Ch]



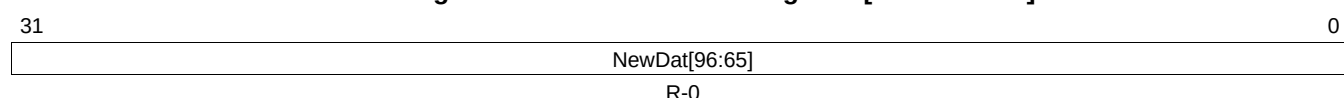
LEGEND: R = Read only; -n = value after reset

Figure 26-35. New Data 34 Register [offset = A0h]



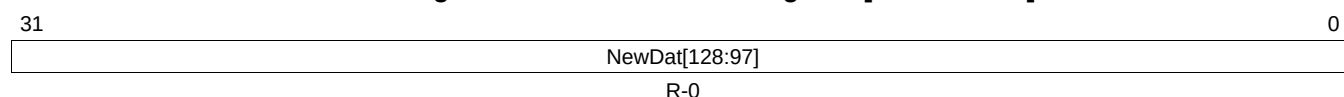
LEGEND: R = Read only; -n = value after reset

Figure 26-36. New Data 56 Register [offset = A4h]



LEGEND: R = Read only; -n = value after reset

Figure 26-37. New Data 78 Register [offset = A8h]



LEGEND: R = Read only; -n = value after reset

Table 26-17. New Data Registers Field Descriptions

Bit	Name	Value	Description
31-0	NewDat[n]	0	New Data Bits (for all message objects) No new data has been written into the data portion of this message object by the Message Handler since the last time when this flag was cleared by the CPU.
		1	The Message Handler or the CPU has written new data into the data portion of this message object.

26.17.14 Interrupt Pending X Register (DCAN INTPND X)

With the Interrupt Pending X Register, the CPU can detect if one or more bits in the different Interrupt Pending Registers are set. Each bit of this register represents a group of eight message objects. If at least one of the IntPnd bits of these message objects are set, the corresponding bit in the Interrupt Pending X Register will be set.

Figure 26-38. Interrupt Pending X Register (DCAN INTPND X) [offset = ACh]

31																16															
Reserved																															
R-0																															
15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0	
IntPndReg8				IntPndReg7				IntPndReg6				IntPndReg5				IntPndReg4				IntPndReg3				IntPndReg2				IntPndReg1			
R-0				R-0				R-0				R-0				R-0				R-0				R-0				R-0			

LEGEND: R = Read only; -n = value after reset

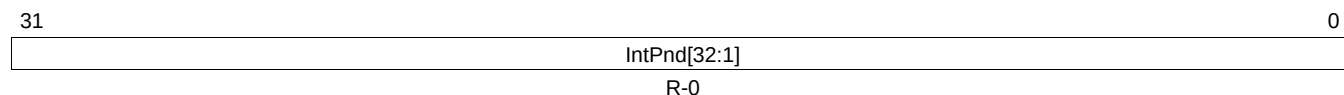
Example 2

Bit 0 of the Interrupt Pending X Register represents byte 0 of the Interrupt Pending 1 Register. If one or more bits in this byte are set, bit 0 of the Interrupt Pending X Register will be set.

26.17.15 Interrupt Pending Registers (DCAN INTPND12 to DCAN INTPND78)

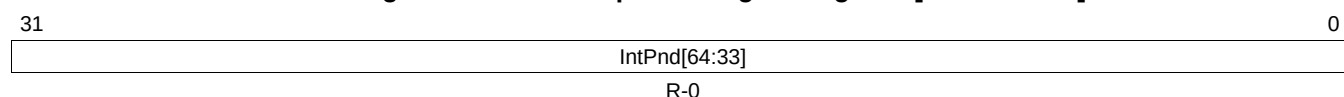
These registers hold the IntPnd bits of the implemented message objects. By reading out these bits, the CPU can check for pending interrupts in the message objects. The IntPnd bit of a specific message object can be set/reset by the CPU via the IF1/IF2 Interface Register sets, or by the Message Handler after a reception or a successful transmission.

Figure 26-39. Interrupt Pending 12 Register [offset = B0h]



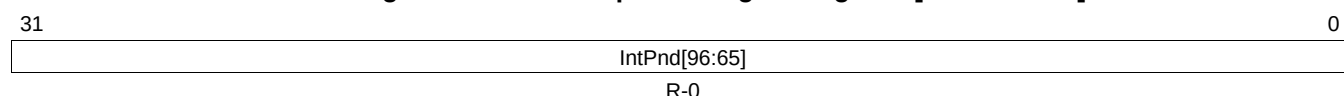
LEGEND: R = Read only; -n = value after reset

Figure 26-40. Interrupt Pending 34 Register [offset = B4h]



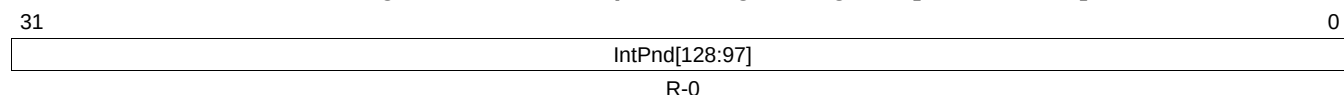
LEGEND: R = Read only; -n = value after reset

Figure 26-41. Interrupt Pending 56 Register [offset = B8h]



LEGEND: R = Read only; -n = value after reset

Figure 26-42. Interrupt Pending 78 Register [offset = BCh]



LEGEND: R = Read only; -n = value after reset

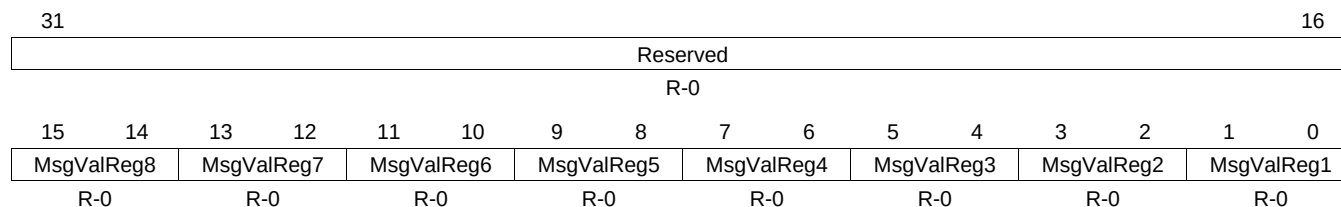
Table 26-18. Interrupt Pending Registers Field Descriptions

Bit	Name	Value	Description
31-0	IntPnd[n]	0	Interrupt Pending Bits (for all message objects)
		0	This message object is not the source of an interrupt.
		1	This message object is the source of an interrupt.

26.17.16 Message Valid X Register (DCAN MSGVAL X)

With the Message Valid X Register, the CPU can detect if one or more bits in the different Message Valid Registers are set. Each bit of this register represents a group of eight message objects. If at least one of the MsgVal bits of these message objects are set, the corresponding bit in the Message Valid X Register will be set.

Figure 26-43. Message Valid X Register (DCAN MSGVAL X) [offset = C0h]



LEGEND: R = Read only; -n = value after reset

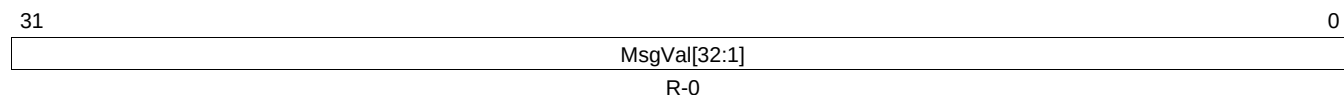
Example 3

Bit 0 of the Message Valid X Register represents byte 0 of the Message Valid 1 Register. If one or more bits in this byte are set, bit 0 of the Message Valid X Register will be set.

26.17.17 Message Valid Registers (DCAN MSGVAL12 to DCAN MSGVAL78)

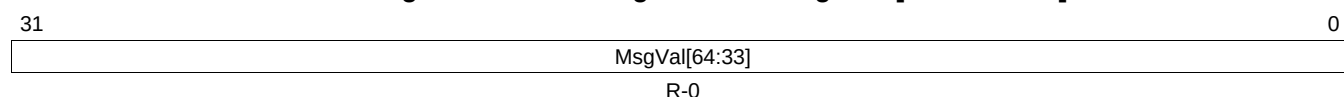
These registers hold the MsgVal bits of the implemented message objects. By reading out these bits, the CPU can check which message objects are valid. The MsgVal bit of a specific message object can be set/reset by the CPU via the IF1/IF2 Interface Register sets, or by the Message Handler after a reception or a successful transmission.

Figure 26-44. Message Valid 12 Register [offset = C4h]



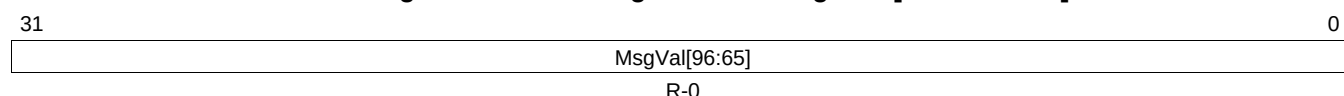
LEGEND: R = Read only; -n = value after reset

Figure 26-45. Message Valid 34 Register [offset = C8h]



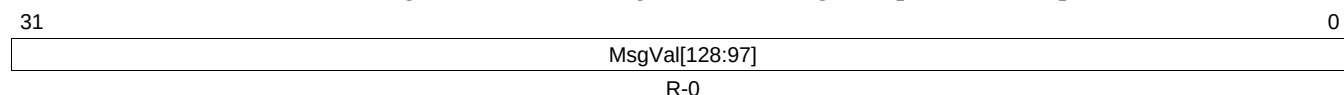
LEGEND: R = Read only; -n = value after reset

Figure 26-46. Message Valid 56 Register [offset = CCh]



LEGEND: R = Read only; -n = value after reset

Figure 26-47. Message Valid 78 Register [offset = D0h]



LEGEND: R = Read only; -n = value after reset

Table 26-19. Message Valid Registers Field Descriptions

Bit	Name	Value	Description
31-0	MsgVal[n]		Message Valid Bits (for all message objects)
		0	This message object is ignored by the Message Handler.
		1	This message object is configured and will be considered by the Message Handler.

26.17.18 Interrupt Multiplexer Registers (DCAN INTMUX12 to DCAN INTMUX78)

The IntMux flag determines for each message object which of the two interrupt lines (DCAN0INT or DCAN1INT) will be asserted when the IntPnd of this message object is set. Both interrupt lines can be globally enabled or disabled by setting or clearing IE0 and IE1 bits in CAN Control Register.

The IntPnd bit of a specific message object can be set or reset by the CPU via the IF1/IF2 Interface Register sets, or by Message Handler after reception or successful transmission of a frame. This will also affect the Int0ID resp Int1ID flags in the Interrupt Register.

Figure 26-48. Interrupt Multiplexer 12 Register [offset = D8h]

31	0
IntMux[32:1]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Figure 26-49. Interrupt Multiplexer 34 Register [offset = DCh]

31	0
IntMux[64:33]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Figure 26-50. Interrupt Multiplexer 56 Register [offset = E0h]

31	0
IntMux[96:65]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Figure 26-51. Interrupt Multiplexer 78 Register [offset = E4h]

31	0
IntMux[128:97]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Table 26-20. Interrupt Multiplexer Registers Field Descriptions

Bit	Name	Value	Description
31-0	IntMux[n]		Multiplexes IntPnd value to either DCAN0INT or DCAN1INT interrupt lines. The mapping from the bits to the message objects is as follows: Bit 0 -> last implemented message object. Bit 1 -> message object number 1. Bit 2 -> message object number 2.
		0	DCAN0INT line is active if corresponding IntPnd flag is 1.
		1	DCAN1INT line is active if corresponding IntPnd flag is 1.

26.17.19 IF1/IF2 Command Registers (DCAN IF1CMD, DCAN IF2CMD)

The IF1/IF2 Command Register configure and Initiate the transfer between the IF1/IF2 Register sets and the Message RAM. It is configurable which portions of the message object should be transferred.

A transfer is started when the CPU writes the message number to bits [7:0] of the IF1/IF2 Command Register. With this write operation, the Busy bit is automatically set to 1 to indicate that a transfer is in progress.

After 4 to 14 VBUS clock cycles, the transfer between the Interface Register and the Message RAM will be completed and the Busy bit is cleared. The maximum number of cycles is needed when the message transfer concurs with a CAN message transmission, acceptance filtering, or message storage.

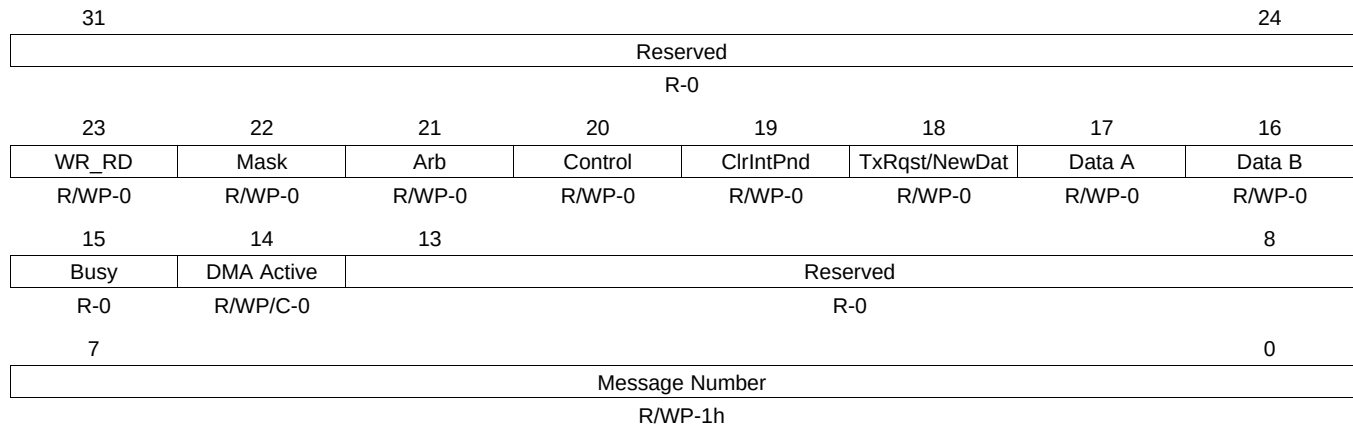
If the CPU writes to both IF1/IF2 Command Registers consecutively (request of a second transfer while first transfer is still in progress), the second transfer will start after the first one has been completed.

NOTE: While the Busy bit is 1, IF1/IF2 Register sets are write protected.

For debug support, the auto clear functionality of the IF1/IF2 Command Registers (clear of DMA Active flag by r/w) is disabled during Debug/Suspend mode.

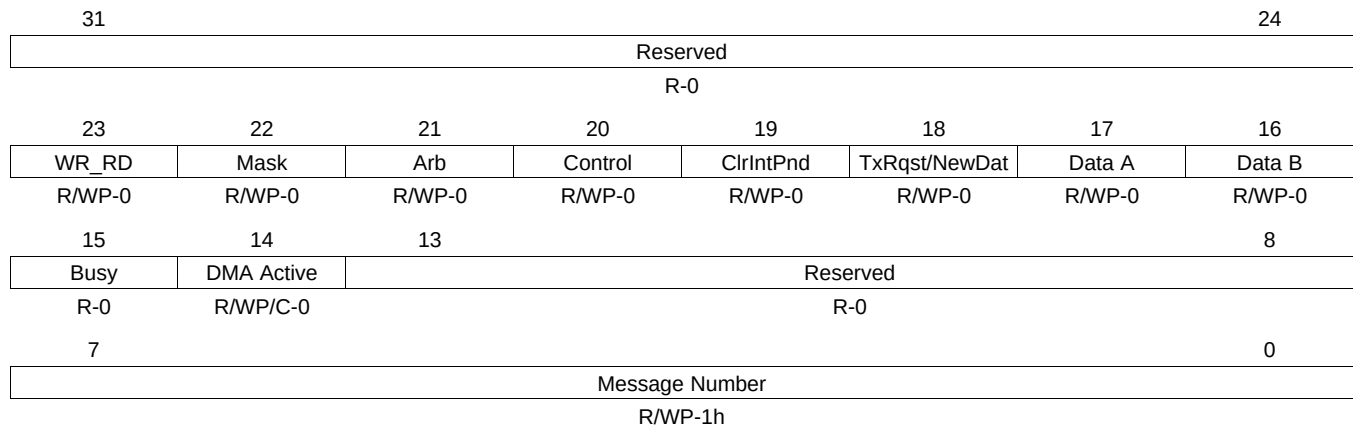
If an invalid Message Number is written to bits [7:0] of the IF1/IF2 Command Register, the Message Handler may access an implemented (valid) message object instead.

Figure 26-52. IF1 Command Registers (DCAN IF1CMD) [offset = 100h]



LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Busy bit); C = Clear by IF1 Access; -n = value after reset

Figure 26-53. IF2 Command Registers (DCAN IF2CMD) [offset = 120h]



LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Busy bit); C = Clear by IF1 Access; -n = value after reset

Table 26-21. IF1/IF2 Command Register Field Descriptions

Bit	Field	Value	Description
31-24	Reserved	0	These bits are always read as 0. Writes have no effect.
23	WR_RD	0 1	Write/Read Direction = Read: Transfer direction is from the message object addressed by Message Number (Bits [7:0]) to the IF1/IF2 register set. Direction = Write: Transfer direction is from the IF1/IF2 register set to the message object addressed by Message Number (Bits [7:0]).
22	Mask	0 1	Access Mask bits Mask bits will not be changed. Direction = Read: The Mask bits (Identifier Mask + MDir + MXtd) will be transferred from the message object addressed by Message Number (Bits [7:0]) to the IF1/IF2 Register set. Direction = Write: The Mask bits (Identifier Mask + MDir + MXtd) will be transferred from the IF1/IF2 Register set to the message object addressed by Message Number (Bits [7:0]).
21	Arb	0 1	Access Arbitration bits Arbitration bits will not be changed. Direction = Read: The Arbitration bits (Identifier + Dir + Xtd + MsgVal) will be transferred from the message object addressed by Message Number (Bits [7:0]) to the corresponding IF1/IF2 Register set. Direction = Write: The Arbitration bits (Identifier + Dir + Xtd + MsgVal) will be transferred from the IF1/IF2 Register set to the message object addressed by Message Number (Bits [7:0]).
20	Control	0 1	Access Control bits Control bits will not be changed Direction = Read: The Message Control bits will be transferred from the message object addressed by Message Number (Bits [7:0]) to the corresponding IF1/IF2 Register set. Direction = Write: The Message Control bits will be transferred from the IF1/IF2 Register set to the message object addressed by Message Number (Bits [7:0]). If the TxRqst/NewDat bit in this register (Bit [18]) is set, the TxRqst/NewDat bit in the IF1/IF2 Message Control Register will be ignored.
19	ClrIntPnd	0 1	Clear Interrupt Pending bit IntPnd bit will not be changed. Direction = Read: Clears IntPnd bit in the message object. Direction = Write: This bit is ignored. Copying of IntPnd flag from IF1/IF2 Registers to Message RAM can be controlled by only the Control flag (Bit [20]).
18	TxRqst/NewDat	0 1	Access Transmission Request bit Direction = Read: NewDat bit will not be changed. Direction = Write: TxRqst/NewDat bit will be handled according to the Control bit. Direction = Read: Clears NewDat bit in the message object. Direction = Write: Sets TxRqst/NewDat in the message object. Note: If a CAN transmission is requested by setting TxRqst/NewDat in this register, the TxRqst/NewDat bits in the message object will be set to 1 and independent of the values in IF1/IF2 Message Control Register. A read access to a message object can be combined with the reset of the control bits IntPnd and NewDat. The values of these bits transferred to the IF1/IF2 Message Control Register always reflect the status before resetting them.
17	Data A	0 1	Access Data Bytes 0-3 Data Bytes 0-3 will not be changed. Direction = Read: The Data Bytes 0-3 will be transferred from the message object addressed by the Message Number (Bits [7:0]) to the corresponding IF1/IF2 Register set Direction = Write: The Data Bytes 0-3 will be transferred from the IF1/IF2 Register set to the message object addressed by the Message Number (Bits [7:0]) Note: The duration of the message transfer is independent of the number of bytes to be transferred.

Table 26-21. IF1/IF2 Command Register Field Descriptions (continued)

Bit	Field	Value	Description
16	Data B	0 1	Access Data Bytes 4-7 Data Bytes 4-7 will not be changed Direction = Read: The Data Bytes 4-7 will be transferred from the message object addressed by the Message Number (Bits [7:0]) to the corresponding IF1/IF2 Register set. Direction = Write: The Data Bytes 4-7 will be transferred from the IF1/IF2 Register set to the message object addressed by the Message Number (Bits [7:0]). Note: The duration of the message transfer is independent of the number of bytes to be transferred.
15	Busy	0 1	Busy flag No transfer between IF1/IF2 Register set and Message RAM is in progress. Transfer between IF1/IF2 Register set and Message RAM is in progress. This bit is set to 1 after the message number has been written to bits [7:0]. IF1/IF2 Register set will be write-protected. The bit is cleared after read/write action has finished.
14	DMA Active	0 1	Activation of DMA feature for subsequent internal IF1/IF2 update DMA request line is independent of IF1/IF2 activities. DMA is requested after completed transfer between IF1/IF2 Register set and Message RAM. The DMA request remains active until the first read or write to one of the IF1/IF2 registers. An exception is a write to Message Number (Bits [7:0]) when DMA Active is 1. Note: Due to the auto reset feature of the DMA Active bit, this bit has to be separately set for each subsequent DMA cycle.
13-8	Reserved	0	These bits are always read as 0. Writes have no effect.
7-0	Message Number	0 1h-40h 41h-FFh	Number of message object in Message RAM that is used for data transfer Invalid message number. Valid message numbers. Invalid message numbers. Note: When an invalid message number is written to the IF1/IF2 Command Register that is higher than the last implemented message object number, a modulo addressing will occur. For example, when accessing message object 33 in a DCAN module with 32 message objects only, the message object 1 will be accessed instead.

26.17.20 IF1/IF2 Mask Registers (DCAN IF1MSK, DCAN IF2MSK)

The bits of the IF1/IF2 Mask Registers mirror the mask bits of a message object. The function of the relevant message objects bits is described in [Section 26.5.1](#).

NOTE: While the Busy bit in the IF1/IF2 Command Register is 1, IF1/IF2 Register Set is write protected.

Figure 26-54. IF1 Mask Register (DCAN IF1MSK) [offset = 104h]

31	30	29	28	16
MXtd	MDir	Rsvd	Msk[28:16]	
R/WP-1	R/WP-1	R-1	R/WP-1FFFh	
15				0
			Msk[15:0]	
			R/WP-FFFFh	

LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Busy bit); -n = value after reset

Figure 26-55. IF2 Mask Register (DCAN IF2MSK) [offset = 124h]

31	30	29	28	16
MXtd	MDir	Rsvd	Msk[28:16]	
R/WP-1	R/WP-1	R-1	R/WP-1FFFh	
15				0
			Msk[15:0]	
			R/WP-FFFFh	

LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Busy bit); -n = value after reset

Table 26-22. IF1/IF2 Mask Register Field Descriptions

Bit	Field	Value	Description
31	MXtd	0 1	Mask Extended Identifier The extended identifier bit (IDE) has no effect on the acceptance filtering. The extended identifier bit (IDE) is used for acceptance filtering. When 11-bit ("standard") identifiers are used for a message object, the identifiers of received Data Frames are written into bits ID[28:18]. For acceptance filtering, only these bits with mask bits Msk[28:18] are considered.
30	MDir	0 1	Mask Message Direction The message direction bit (Dir) has no effect on the acceptance filtering. The message direction bit (Dir) is used for acceptance filtering.
29	Reserved	1	This bit is always read as 1. Writes have no effect.
28-0	Msk[n]	0 1	Identifier Mask The corresponding bit in the identifier of the message object is not used for acceptance filtering (don't care). The corresponding bit in the identifier of the message object is used for acceptance filtering.

26.17.21 IF1/IF2 Arbitration Registers (DCAN IF1ARB, DCAN IF2ARB)

The bits of the IF1/IF2 Arbitration Registers mirror the arbitration bits of a message object. The function of the relevant message objects bits is described in [Section 26.5.1](#).

NOTE: While the Busy bit in the IF1/IF2 Command Register is 1, IF1/IF2 Register Set is write protected.

Figure 26-56. IF1 Arbitration Register (DCAN IF1ARB) [offset = 108h]

31	30	29	28	16
MsgVal	Xtd	Dir	ID[28:16]	
R/WP-0	R/WP-0	R/WP-0	R/WP-0	
15				0
			ID[15:0]	
			R/WP-0	

LEGEND: R/W = Read/Write; WP = Protected Write (protected by Busy bit); -n = value after reset

Figure 26-57. IF2 Arbitration Register (DCAN IF2ARB) [offset = 128h]

31	30	29	28	16
MsgVal	Xtd	Dir	ID[28:16]	
R/WP-0	R/WP-0	R/WP-0	R/WP-0	
15				0
			ID[15:0]	
			R/WP-0	

LEGEND: R/W = Read/Write; WP = Protected Write (protected by Busy bit); -n = value after reset

Table 26-23. IF1/IF2 Arbitration Register Field Descriptions

Bit	Field	Value	Description
31	MsgVal	0 1	Message Valid The message object is ignored by the Message Handler. The message object is used by the Message Handler. Note: The CPU should reset the MsgVal bit of all unused Messages Objects during the initialization before it resets bit Init in the CAN Control Register. MsgVal must also be reset if the messages object is no longer used in operation. For reconfiguration of message objects during normal operation, see Section 26.7.6 and Section 26.7.7 .
30	Xtd	0 1	Extended Identifier The 11-bit ("standard") identifier is used for this message object. The 29-bit ("extended") identifier is used for this message object.
29	Dir	0 1	Message direction Direction = Receive: On TxRqst, a Remote Frame with the identifier of this message object is transmitted. On receiving a Data Frame with a matching identifier, this message is stored in this message object. Direction = Transmit: On TxRqst, the respective message object is transmitted as a Data Frame. On receiving a Remote Frame with a matching identifier, the TxRqst bit of this message object is set (if RmtEn = 1).
28-0	ID[28:0]	ID[28:0] ID[28:18]	Message Identifier 29-bit Identifier ("Extended Frame") 11-bit Identifier ("Standard Frame")

The Arbitration bits ID[28:0], Xtd, and Dir are used to define the identifier and type of outgoing messages and (together with the Mask bits Msk[28:0], MXtd, and MDir) for acceptance filtering of incoming messages.

A received message is stored into the valid message object with matching identifier and Direction = receive (Data Frame) or Direction = transmit (Remote Frame).

Extended frames can be stored only in message objects with Xtd = 1, standard frames in message objects with Xtd = 0.

If a received message (Data Frame or Remote Frame) matches more than one valid message objects, it is stored into the one with the lowest message number.

26.17.22 IF1/IF2 Message Control Registers (DCAN IF1MCTL, DCAN IF2MCTL)

The bits of the IF1/IF2 Message Control Registers mirror the message control bits of a message object. The function of the relevant message objects bits is described in [Section 26.5.1](#).

NOTE: While the Busy bit in the IF1/IF2 Command Register is 1, IF1/IF2 Register Set is write protected.

Figure 26-58. IF1 Message Control Register (DCAN IF1MCTL) [offset = 10Ch]

31																16																			
Reserved																																			
R-0																																			
15				14				13				12				11				10				9				8							
NewDat				MsgLst				IntPnd				UMask				TxIE				RxIE				RmtEn				TxRqst							
R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0							
7				6				4				3												0											
EoB				Reserved																DLC															
R/WP-0				R-0																R/WP-0															

LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Busy bit); -n = value after reset

Figure 26-59. IF2 Message Control Register (DCAN IF2MCTL) [offset = 12Ch]

31																16																																															
Reserved																																																															
R-0																																																															
15								14								13								12								11								10								9								8							
NewDat				MsgLst				IntPnd				UMask				TxIE				RxIE				RmtEn				TxRqst																																			
R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0				R/WP-0																															
7								6								4								3								0																															
EoB				Reserved																DLC																																											
R/WP-0				R-0																R/WP-0																																											

LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Busy bit); -n = value after reset

Table 26-24. IF1/IF2 Message Control Register Field Descriptions

Bit	Field	Value	Description
31-16	Reserved		These bits are always read as 0. Writes have no effect.
15	NewDat	0 1	New Data No new data has been written into the data portion of this message object by the Message Handler since the last time this flag was cleared by the CPU. The Message Handler or the CPU has written new data into the data portion of this message object.
14	MsgLst	0 1	Message Lost (only valid for message objects with direction = receive) No message lost since the last time when this bit was reset by the CPU. The Message Handler stored a new message into this object when NewDat was still set, so the previous message has been overwritten.
13	IntPnd	0 1	Interrupt Pending This message object is not the source of an interrupt. This message object is the source of an interrupt. The interrupt identifier in the interrupt register will point to this message object if there is no other interrupt source with higher priority.
12	UMask	0 1	Use Acceptance Mask Mask ignored Use Mask (Msk[28:0], MXtd, and MDir) for acceptance filtering. If the UMask bit is set to 1, the message object's mask bits have to be programmed during initialization of the message object before MsgVal is set to 1.
11	TxE	0 1	Transmit Interrupt Enable IntPnd will not be triggered after the successful transmission of a frame. IntPnd will be triggered after the successful transmission of a frame.
10	RxE	0 1	Receive Interrupt Enable IntPnd will not be triggered after the successful reception of a frame. IntPnd will be triggered after the successful reception of a frame.
9	RmtEn	0 1	Remote Enable At the reception of a Remote Frame, TxRqst is not changed. At the reception of a Remote Frame, TxRqst is set.
8	TxRqst	0 1	Transmit Request This message object is not waiting for a transmission. The transmission of this message object is requested and not yet done.
7	EoB	0 1	End of Block The message object is part of a FIFO Buffer block and is not the last message object of this FIFO Buffer block. The message object is a single message object or the last message object in a FIFO Buffer block. Note: This bit is used to concatenate multiple message objects to build a FIFO Buffer. For single message objects (not belonging to a FIFO Buffer), this bit must always be set to 1.
6-4	Reserved		These bits are always read as 0. Writes have no effect.
3-0	DLC	0-8h 9h-Fh	Data Length Code Data Frame has 0-8 data bytes. Data Frame has 8 data bytes. Note: The Data Length Code of a message object must be defined the same as in all the corresponding objects with the same identifier at other nodes. When the message handler stores a data frame, it will write the DLC to the value given by the received message.

26.17.23 IF1/IF2 Data A and Data B Registers (DCAN IF1DATA/DATB, DCAN IF2DATA/DATB)

The data bytes of CAN messages are stored in the IF1/IF2 registers in the following order.

In a CAN Data Frame, Data 0 is the first, and Data 7 is the last byte to be transmitted or received. In CAN's serial bit stream, the MSB of each byte will be transmitted first

Figure 26-60. IF1 Data A Register (DCAN IF1DATA) [offset = 110h]

31	24	23	16
Data 3		Data 2	
R/WP-0		R/WP-0	
15	8	7	0
Data 1		Data 0	
R/WP-0		R/WP-0	

LEGEND: R/W = Read/Write; WP = Protected Write (protected by Busy bit); -n = value after reset

Figure 26-61. IF1 Data B Register (DCAN IF1DATB) [offset = 114h]

31	24	23	16
Data 7		Data 6	
R/WP-0		R/WP-0	
15	8	7	0
Data 5		Data 4	
R/WP-0		R/WP-0	

LEGEND: R/W = Read/Write; WP = Protected Write (protected by Busy bit); -n = value after reset

Figure 26-62. IF2 Data A Register (DCAN IF2DATA) [offset = 130h]

31	24	23	16
Data 3		Data 2	
R/WP-0		R/WP-0	
15	8	7	0
Data 1		Data 0	
R/WP-0		R/WP-0	

LEGEND: R/W = Read/Write; WP = Protected Write (protected by Busy bit); -n = value after reset

Figure 26-63. IF2 Data B Register (DCAN IF2DATB) [offset = 134h]

31	24	23	16
Data 7		Data 6	
R/WP-0		R/WP-0	
15	8	7	0
Data 5		Data 4	
R/WP-0		R/WP-0	

LEGEND: R/W = Read/Write; WP = Protected Write (protected by Busy bit); -n = value after reset

26.17.24 IF3 Observation Register (DCAN IF3OBS)

The IF3 register set can automatically be updated with received message objects without the need to Initiate the transfer from Message RAM by CPU (Additional information can be found in [Section 26.5.1](#)).

The observation flags (bits [4:0]) in the IF3 Observation register are used to determine which data sections of the IF3 Interface Register set have to be read in order to complete a DMA read cycle. After all marked data sections are read, the DCAN is enabled to update the IF3 Interface Register set with new data.

Any access order of single bytes or half-words is supported. When using byte or half-word accesses, a data section is marked as completed, if all bytes are read.

NOTE: If IF3 Update Enable is used and no Observation flag is set, the corresponding message objects will be copied to IF3 without activating the DMA request line and without waiting for DMA read accesses.

A write access to this register aborts a pending DMA cycle by resetting the DMA line and enables updating of IF3 Interface Register set with new data. To avoid data inconsistency, the DMA controller should be disabled before reconfiguring IF3 observation register.

The status of the current read-cycle can be observed via status flags (Bits [12:8]).

An interrupt request may be generated by the IF3Upd flag if the DE3 bit of DCAN CTL register is set. See the device data sheet to find out if this interrupt source is available.

With this, the observation status bits and the IF3Upd bit could be used by the application to realize the notification about new IF3 content in polling or interrupt mode.

Figure 26-64. IF3 Observation Register (DCAN IF3OBS) [offset = 140h]

31																16																																															
Reserved																																																															
R-0																																																															
15								14								13								12								11								10								9								8							
IF3Upd				Reserved												IF3SDB				IF3SDA				IF3SC				IF3SA				IF3SM																															
R-0								R-0								R-0								R-0								R-0								R-0																							
7								5								4								3								2								1								0															
Reserved																DataB				DataA				Ctrl				Arb				Mask																															
R-0																R/W-0								R/W-0								R/W-0								R/W-0								R/W-0															

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 26-25. IF3 Observation Register Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	These bits are always read as 0. Writes have no effect.
15	IF3Upd	0 1	IF3 Update Data No new data has been loaded since IF3 was last read. New data has been loaded since IF3 was last read.
14-13	Reserved	0	These bits are always read as 0. Writes have no effect
12	IF3SDB	0 1	IF3 Status of Data B read access All Data B bytes are already read or are not marked to be read. Data B section still has data to read.
11	IF3SDA	0 1	IF3 Status of Data A read access All Data A bytes are already read or are not marked to be read. Data A section still has data to read.
10	IF3SC	0 1	IF3 Status of Control bits read access All Control section bytes are already read or are not marked to be read. Control section still has data to read.

Table 26-25. IF3 Observation Register Field Descriptions (continued)

Bit	Field	Value	Description
9	IF3SA	0	IF3 Status of Arbitration data read access All Arbitration data bytes are already read or are not marked to be read.
		1	Arbitration section still has data to read.
8	IF3SM	0	IF3 Status of Mask data read access All Mask data bytes are already read or are not marked to be read.
		1	Mask section still has data to read.
7-5	Reserved	0	These bits are always read as 0. Writes have no effect
4	DataB	0	Data B read observation Data B section does not need to be read.
		1	Data B section has to be read to enable next IF3 update.
3	DataA	0	Data A read observation Data A section does not need to be read.
		1	Data A section has to be read to enable next IF3 update.
2	Ctrl	0	Ctrl read observation Ctrl section does not need to be read.
		1	Ctrl section has to be read to enable next IF3 update.
1	Arb	0	Arbitration data read observation Arbitration data does not need to be read.
		1	Arbitration data has to be read to enable next IF3 update.
0	Mask	0	Mask data read observation Mask data does not need to be read.
		1	Mask data has to be read to enable next IF3 update.

26.17.25 IF3 Mask Register (DCAN IF3MSK)

Figure 26-65. IF3 Mask Register (DCAN IF3MSK) [offset = 144h]

31	30	29	28	16
MXtd	MDir	Rsvd	Msk[28:16]	
R-1	R-1	R-1	R-1FFFh	
15				0
			Msk[15:0]	
			R-FFFFh	

LEGEND: R = Read only; -n = value after reset

Table 26-26. IF3 Mask Register Field Descriptions

Bit	Field	Value	Description
31	MXtd	0 1	Mask Extended Identifier The extended identifier bit (IDE) has no effect on acceptance filtering. The extended identifier bit (IDE) is used for acceptance filtering. Note: When 11-bit ("standard") identifiers are used for a message object, the identifiers of received Data Frames are written into bits ID[28:18]. For acceptance filtering, only these bits, together with mask bits Msk[28:18], are considered.
30	MDir	0 1	Mask Message Direction The message direction bit (Dir) has no effect on acceptance filtering. The message direction bit (Dir) is used for acceptance filtering.
29	Reserved	1	This bit is always read as 1. Writes have no effect.
28-0	Msk[n]	0 1	Identifier Mask The corresponding bit in the identifier of the message object is not used for acceptance filtering (don't care). The corresponding bit in the identifier of the message object is used for acceptance filtering.

26.17.26 IF3 Arbitration Register (DCAN IF3ARB)

Figure 26-66. IF3 Arbitration Register (DCAN IF3ARB) [offset = 148h]

31	30	29	28	16
MsgVal	Xtd	R-0	ID[28:16]	
R-0	R-0	R-0	R-0	
15				0
			ID[15:0]	
			R-0	

LEGEND: R = Read only; -n = value after reset

Table 26-27. IF3 Arbitration Register Field Descriptions

Bit	Field	Value	Description
31	MsgVal	0 1	Message Valid The message object is ignored by the Message Handler. The message object is to be used by the Message Handler. Note: The CPU should reset the MsgVal bit of all unused Messages Objects during the initialization before it resets bit Init in the CAN Control Register. MsgVal must also be reset if the messages object is no longer used in operation. For reconfiguration of message objects during normal operation, see Section 26.7.6 and Section 26.7.7.
30	Xtd	0 1	Extended Identifier The 11-bit ("standard") identifier is used for this message object. The 29-bit ("extended") identifier is used for this message object.
29	Dir	0 1	Message direction Direction = Receive: On TxRqst, a remote frame with the identifier of this message object is transmitted. On receiving a data frame with a matching identifier, the message is stored in this message object. Direction = Transmit: On TxRqst, the respective message object is transmitted as a data frame. On receiving a remote frame with a matching identifier, the TxRqst bit of this message object is set (if RmtEn = 1).
28-0	ID[28:0]	ID[28:0] ID[28:18]	Message Identifier 29-bit Identifier ("Extended Frame") 11-bit Identifier ("Standard Frame")

26.17.27 IF3 Message Control Register (DCAN IF3MCTL)

Figure 26-67. IF3 Message Control Register (DCAN IF3MCTL) [offset = 14Ch]

31																16																																															
Reserved																																																															
R-0																																																															
15								14								13								12								11								10								9								8							
NewDat				MsgLst				IntPnd				UMask				TxIE				RxIE				RmtEn				TxRqst																																			
R-0								R-0								R-0								R-0								R-0								R-0								R-0															
7								6								4								3																0																							
EoB				Reserved																DLC																																											
R-0								R-0																R-0																																							

LEGEND: R = Read only; -n = value after reset

Table 26-28. IF3 Message Control Register Field Descriptions

Bit	Field	Value	Description
31-16	Reserved	0	These bits are always read as 0. Writes have no effect.
15	NewDat		New Data
		0	No new data has been written into the data portion of this message object by the Message Handler since the last time this flag was cleared by the CPU.
		1	The Message Handler or the CPU has written new data into the data portion of this message object.
14	MsgLst		Message Lost (only valid for message objects with direction = receive)
		0	No message lost since the last time when this bit was reset by the CPU.
		1	The Message Handler stored a new message into this object when NewDat was still set, so the previous message has been overwritten.
13	IntPnd		Interrupt Pending
		0	This message object is not the source of an interrupt.
		1	This message object is the source of an interrupt. The interrupt identifier in the interrupt register will point to this message object if there is no other interrupt source with higher priority.
12	UMask		Use Acceptance Mask
		0	Mask ignored
		1	Use Mask (Msk[28:0], MXtd, and MDir) for acceptance filtering. If the UMask bit is set to 1, the message object's mask bits have to be programmed during initialization of the message object before MsgVal is set to 1.
11	TxIE		Transmit Interrupt Enable
		0	IntPnd will not be triggered after the successful transmission of a frame.
		1	IntPnd will be triggered after the successful transmission of a frame.
10	RxIE		Receive Interrupt Enable
		0	IntPnd will not be triggered after the successful transmission of a frame.
		1	IntPnd will be triggered after the successful transmission of a frame.
9	RmtEn		Remote Enable
		0	At the reception of a Remote Frame, TxRqst is not changed.
		1	At the reception of a Remote Frame, TxRqst is set.
8	TxRqst		Transmit Request
		0	This message object is not waiting for a transmission.
		1	The transmission of this message object is requested and not yet done.

Table 26-28. IF3 Message Control Register Field Descriptions (continued)

Bit	Field	Value	Description
7	EoB	0 1	End of Block The message object is part of a FIFO Buffer block and is not the last message object of the FIFO Buffer block. The message object is a single message object or the last message object in a FIFO Buffer block. Note: This bit is used to concatenate multiple message objects to build a FIFO Buffer. For single message objects (not belonging to a FIFO Buffer), this bit must always be set to 1.
6-4	Reserved	0	These bits are always read as 0. Writes have no effect.
3-0	DLC	0-8h 9h-Fh	Data Length Code Data Frame has 0-8 data bytes. Data Frame has 8 data bytes. Note: The Data Length Code of a message object must be defined the same as in all the corresponding objects with the same identifier at other nodes. When the message handler stores a data frame, it will write the DLC to the value given by the received message.

26.17.28 IF3 Data A and Data B Registers (DCAN IF3DATA/DATB)

The data bytes of CAN messages are stored in the IF3 registers in the following order.

In a CAN Data Frame, Data 0 is the first, and Data 7 is the last byte to be transmitted or received. In CAN's serial bit stream, the MSB of each byte will be transmitted first.

Figure 26-68. IF3 Data A Register (DCAN IF3DATA) [offset = 150h]

31	24	23	16
Data 3		Data 2	
R-0		R-0	
15	8	7	0
Data 1		Data 0	
R-0		R-0	

LEGEND: R = Read only; -n = value after reset

Figure 26-69. IF3 Data B Register (DCAN IF3DATB) [offset = 154h]

31	24	23	16
Data 7		Data 6	
R/WP-0		R/WP-0	
15	8	7	0
Data 5		Data 4	
R-0		R-0	

LEGEND: R = Read only; -n = value after reset

26.17.29 IF3 Update Enable Registers (DCAN IF3UPD12 to IF3UPD78)

The automatic update functionality of the IF3 register set can be configured for each message object. A message object is enabled for automatic IF3 update, if the dedicated IF3UpdEn flag is set. This means that an active NewDat flag of this message object (for example, due to reception of a CAN frame) will trigger an automatic copy of the whole message object to IF3 register set.

NOTE: IF3 Update enable should not be set for transmit objects.

Figure 26-70. IF3 Update Enable 12 Register [offset = 160h]

31	0
IF3UpdEn[32:1]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Figure 26-71. IF3 Update Enable 34 Register [offset = 164h]

31	0
IF3UpdEn[64:33]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Figure 26-72. IF3 Update Enable 56 Register [offset = 168h]

31	0
IF3UpdEn[96:65]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Figure 26-73. IF3 Update Enable 78 Register [offset = 16Ch]

31	0
IF3UpdEn[128:97]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value after reset

Table 26-29. IF3 Update Control Register Field Descriptions

Bit	Name	Value	Description
31-0	IF3UpdEn[n]		IF3 Update Enabled (for all message objects)
		0	Automatic IF3 update is disabled for this message object.
		1	Automatic IF3 update is enabled for this message object. A message object is scheduled to be copied to IF3 register set, if NewDat flag of the message object is active.

26.17.30 CAN TX IO Control Register (DCAN TIOC)

The CAN_TX pin of the DCAN module can be used as general-purpose IO pin if CAN function is not needed.

NOTE: The values of the IO Control registers are only writable if Init bit of CAN Control Register is set.

The OD, Func, Dir, and Out bits of the CAN TX IO Control register are forced to certain values when Init bit of CAN Control Register is reset (see bit descriptions).

Figure 26-74. CAN TX IO Control Register (DCAN TIOC) [offset = 1E0h]

31				19	18	17	16	
Reserved					PU	PD	OD	
R-0					R/W-D	R/W-D	R/WP-0	
15				4	3	2	1	0
Reserved					Func	Dir	Out	In
R-0					R/WP-0	R/WP-0	R/WP-0	R-U

LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Init bit); D = Device-dependent; -n = value after reset; U = Undefined

Table 26-30. CAN TX IO Control Register Field Descriptions

Bit	Field	Value	Description
31-19	Reserved	0	These bits are always read as 0. Writes have no effect.
18	PU	0 1	CAN_TX Pullup/Pulldown select. This bit is only active when CAN_TX is configured to be an input. 0 CAN_TX Pulldown is selected, when pull logic is active (PD = 0). 1 CAN_TX Pullup is selected, when pull logic is active (PD = 0).
17	PD	0 1	CAN_TX pull disable. This bit is only active when CAN_TX is configured to be an input. 0 CAN_TX pull is active. 1 CAN_TX pull is disabled.
16	OD	0 1	CAN_TX open drain enable. This bit is only active when CAN_TX is configured to be in GIO mode (TIOC.Func = 0). 0 The CAN_TX pin is configured in push/pull mode. 1 The CAN_TX pin is configured in open drain mode. Forced to 0 if Init bit of CAN control register is reset.
15-4	Reserved	0	These bits are always read as 0. Writes have no effect.
3	Func	0 1	CAN_TX function. This bit changes the function of the CAN_TX pin. 0 CAN_TX pin is in GIO mode. 1 CAN_TX pin is in functional mode (as an output to transmit CAN data). Forced to 1 if Init bit of CAN control register is reset.
2	Dir	0 1	CAN_TX data direction. This bit controls the direction of the CAN_TX pin when it is configured to be in GIO mode only (TIOC.Func = 0). 0 The CAN_TX pin is an input. 1 The CAN_TX pin is an output. Forced to 1 if Init bit of CAN control register is reset.
1	Out	0 1	CAN_TX data out write. This bit is only active when CAN_TX pin is configured to be in GIO mode (TIOC.Func = 0) and configured to be an output pin (TIOC.Dir = 1). The value of this bit indicates the value to be output to the CAN_TX pin. 0 The CAN_TX pin is driven to logic low (0). 1 The CAN_TX pin is at logic high (1). Forced to Tx output of the CAN Core, if Init bit of CAN Control register is reset.

Table 26-30. CAN TX IO Control Register Field Descriptions (continued)

Bit	Field	Value	Description
0	In	0 1	CAN_TX data in. The CAN_TX pin is at logic low (0). The CAN_TX pin is at logic high (1). Note: When CAN_TX pin is connected to a CAN transceiver, an external pullup resistor has to be used to ensure that the CAN bus will not be disturbed (for example, while the DCAN module is reset).

26.17.31 CAN RX IO Control Register (DCAN RIOC)

The CAN_RX pin of the DCAN module can be used as general-purpose IO pin if CAN function is not needed.

NOTE: The values of the IO Control registers are writable only if Init bit of CAN Control Register is set.

The OD, Func, and Dir bits of the CAN RX IO Control register are forced to certain values when Init bit of CAN Control Register is reset, see bit description.

Figure 26-75. CAN RX IO Control Register (DCAN RIOC) [offset = 1E4h]

31				19	18	17	16	
Reserved					PU	PD	OD	
R-0					R/W-D	R/W-D	R/WP-0	
15				4	3	2	1	0
Reserved					Func	Dir	Out	In
R-0					R/WP-0	R/WP-0	R/WP-0	R-U

LEGEND: R/W = Read/Write; R = Read only; WP = Protected Write (protected by Init bit); D = Device-dependent; -n = value after reset; U = Undefined

Table 26-31. CAN RX IO Control Register Field Descriptions

Bit	Field	Value	Description
31-19	Reserved	0	These bits are always read as 0. Writes have no effect.
18	PU	0 1	CAN_RX Pullup/Pulldown select. This bit is only active when CAN_RX is configured to be an input. CAN_RX Pulldown is selected, when pull logic is active (PD = 0). CAN_RX Pullup is selected, when pull logic is active (PD = 0).
17	PD	0 1	CAN_RX pull disable. This bit is only active when CAN_RX is configured to be an input. CAN_RX pull is active. CAN_RX pull is disabled.
16	OD	0 1	CAN_RX open drain enable. This bit is only active when CAN_RX is configured to be in GIO mode (RIOC.Func = 0). The CAN_RX pin is configured in push/pull mode. The CAN_RX pin is configured in open drain mode. Forced to 0 if Init bit of CAN control register is reset.
15-4	Reserved	0	These bits are always read as 0. Writes have no effect.
3	Func	0 1	CAN_RX function. This bit changes the function of the CAN_RX pin. CAN_RX pin is in GIO mode. CAN_RX pin is in functional mode (as an input to receive CAN data). Forced to 1 if Init bit of CAN control register is reset.

Table 26-31. CAN RX IO Control Register Field Descriptions (continued)

Bit	Field	Value	Description
2	Dir	0 1	CAN_RX data direction. This bit controls the direction of the CAN_RX pin when it is configured to be in GIO mode only (RIOCFunc = 0). The CAN_RX pin is an input. The CAN_RX pin is an output. Forced to 0 if Init bit of CAN control register is reset.
1	Out	0 1	CAN_RX data out write. This bit is only active when CAN_RX pin is configured to be in GIO mode (RIOCFunc = 0) and configured to be an output pin (RIOCDir = 1). The value of this bit indicates the value to be output to the CAN_RX pin. The CAN_RX pin is driven to logic low (0). The CAN_RX pin is at logic high (1).
0	In	0 1	CAN_RX data in. The CAN_RX pin is at logic low (0). The CAN_RX pin is at logic high (1). Note: When CAN_RX pin is connected to a CAN transceiver, an external pullup resistor has to be used to ensure that the CAN bus will not be disturbed (for example, while the DCAN module is reset).